# FEATURE DRIFT DETECTION VIA ADVERSARIAL VALIDATION

**NUSRAT BEGUM**

**A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE (COMPUTER SCIENCE)
FACULTY OF GRADUATE STUDIES
MAHIDOL UNIVERSITY
2026**

Thesis
entitled

# FEATURE DRIFT DETECTION VIA ADVERSARIAL VALIDATION

.....
Ms. Nusrat Begum
Candidate

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Major advisor

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Co-advisor

.....
Assistant Professor Dr.Warin Krityakiarana
Deputy Dean for Planning and Quality Assurance
Acting as Dean of the Faculty of Graduate Studies, Mahidol University
Dean
Faculty of Graduate Studies
Mahidol University

.....
Assoc. Prof. Boonsit Yimwadsana,
Ph.D. (Electrical Engineering)
Program Director
Master of Science Programme in Computer Science
Faculty of Information and Communication Technology
Mahidol University

Thesis
entitled

# FEATURE DRIFT DETECTION VIA ADVERSARIAL VALIDATION

was submitted to the Faculty of Graduate Studies, Mahidol University
for the degree of Master of Science (Computer Science)
on
February 2026

.....
Ms. Nusrat Begum
Candidate

.....
Assistant Professor Dr. Akara Supratak,
Ph.D. (Computer Science)
Chair

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Member

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Member

.....
Assistant Professor Dr.Warin Krityakiarana
Deputy Dean for Planning and Quality Assurance
Acting as Dean of the Faculty of Graduate Studies, Mahidol University
Dean
Faculty of Graduate Studies
Mahidol University

.....
Dr. Pattanasak Mongkolwat,
Ph.D. (Computer Science)
Dean
Faculty of Information and Communication Technology
Mahidol University

# ACKNOWLEDGEMENTS

Nusrat Begum

FEATURE DRIFT DETECTION VIA ADVERSARIAL VALIDATION

NUSRAT BEGUM 6638009 ITCS/M

M.Sc. (COMPUTER SCIENCE)

THESIS ADVISORY COMMITTEE: THANAPON NORASET, Ph.D., SUPPAWONG TUAROB, Ph.D.

ABSTRACT

Machine learning models deployed in production environments frequently experience distribution drift—changes in input feature distributions that degrade model performance. While discriminative methods such as D3 have demonstrated that adversarial validation can accurately detect drift, they provide no insight into which features drifted or what corrective action is appropriate.

This thesis proposes Explainable Adversarial Drift Detection (EADD), a framework that extends adversarial validation with SHAP-based root cause analysis and automated remediation prescriptions. EADD employs LightGBM as the adversarial classifier with reservoir sampling for representative reference windows, a permutation test ($B = 50$, $\alpha = 0.01$) for statistically rigorous drift confirmation, and TreeSHAP feature attribution to identify which specific features drive detected drift. The framework classifies drift patterns into univariate, feature-subset, or multivariate categories with corresponding actionable recommendations.

Experimental evaluation demonstrated that EADD detected all four temporal drift types (abrupt, gradual, incremental, recurring) with 100% success rate, whereas D3 detected only two. Across 13 real-world benchmark datasets, EADD achieved 0% missed detection rate with 3.7% lower mean time to detection than D3. On stable data, EADD produced zero false alarms, including on autocorrelated streams where D3 triggered up to 87.4 false alarms (Mann–Whitney $p = 0.0101$). SHAP-based feature attribution correctly identified drifting features in all controlled scenarios (AUC 0.767–0.801). EADD advances drift detection from a binary alarm to an actionable diagnostic tool, addressing the critical explainability gap in modern MLOps pipelines.

IMPLICATION OF THE THESIS

EADD provides both broad drift type coverage and actionable explanations, enabling MLOps teams to respond appropriately to different drift scenarios rather than treating all drift signals as equivalent alarms requiring full model retraining.

KEY WORDS : DRIFT DETECTION / ADVERSARIAL VALIDATION / COVARIATE DRIFT / MLOPS / DISTRIBUTION SHIFT / STREAMING DATA

67 pages

# CONTENTS


| | | Page |
|---|---|---|
| **ACKNOWLEDGEMENTS** | | **iii** |
| **ABSTRACT** | | **iv** |
| **LIST OF TABLES** | | **x** |
| **LIST OF FIGURES** | | **xi** |
| **CHAPTER I** | **INTRODUCTION** | **1** |
| 1.1 | Motivation | 1 |
| 1.2 | Objectives | 3 |
| 1.3 | Expected Contributions | 3 |
| **CHAPTER II** | **LITERATURE REVIEW** | **5** |
| 2.1 | Overview | 5 |
| 2.2 | Types of Drift | 6 |
| 2.2.1 | Drift by Nature of Change | 6 |
| 2.2.2 | Drift by Temporal Pattern | 7 |
| 2.3 | Classification of Drift Detection Methods | 7 |
| 2.3.1 | Supervised Drift Detection Methods | 8 |
| 2.3.2 | Unsupervised Drift Detection Methods | 9 |
| 2.3.3 | Hybrid and Ensemble Methods | 14 |
| 2.4 | Limitations of Existing Methods | 15 |
| 2.5 | Related Work: Adversarial Validation | 16 |
| 2.5.1 | Concept and Origins | 16 |
| 2.5.2 | Industrial Applications | 16 |
| 2.6 | Gap in the Literature | 18 |
| **CHAPTER III** | **METHODOLOGY** | **20** |
| 3.1 | Overview: Explainable Adversarial Drift Detection (EADD) | 20 |
| 3.2 | Problem Formulation | 21 |
| 3.3 | EADD Framework Components | 22 |

# CONTENTS (cont.)


| | | Page |
|---|---|---|
| 3.3.1 | Step 1: Adaptive Reference Windowing | 23 |
| 3.3.2 | Step 2: Adversarial Validation (The Detector) | 23 |
| 3.3.3 | Step 3: Statistical Significance via Permutation Test | 24 |
| 3.3.4 | Step 4: Feature Drift Explainability via SHAP (Novel Contribution) | 25 |
| 3.3.5 | Automated Drift Prescription (Novel Contribution) | 26 |
| 3.3.6 | Model Adaptation Strategies | 27 |
| 3.4 | Datasets | 28 |
| 3.5 | Baseline Drift Detectors | 29 |
| 3.6 | Evaluation Metrics | 30 |
| 3.7 | Experimental Design | 31 |
| 3.8 | Practical Considerations | 32 |
| **CHAPTER IV** | **EXPERIMENTS AND RESULTS** | **34** |
| 4.1 | Research Questions | 34 |
| 4.2 | Hypotheses | 34 |
| 4.3 | Experimental Setup | 35 |
| 4.4 | Experiment 1: Sensitivity to Temporal Drift Patterns | 36 |
| 4.5 | Experiment 2: Real-World Benchmark | 37 |
| 4.6 | Experiment 3: Explainability Case Study | 39 |
| 4.7 | Experiment 4: Robustness to Stable Data (False Alarm Evaluation) | 40 |
| 4.8 | Statistical Hypothesis Testing | 41 |
| 4.8.1 | H1: Detection Coverage and Delay | 41 |
| 4.8.2 | H2: False Alarm Reduction (Mann–Whitney U Test) | 42 |
| 4.8.3 | H3: Explainability Accuracy (Qualitative Evaluation) | 42 |
| 4.9 | Summary of Experimental Results | 43 |
| **CHAPTER V** | **DISCUSSION** | **44** |
| 5.1 | Addressing Core Research Questions | 44 |

# CONTENTS (cont.)


| | | Page |
|---|---|---|
| 5.1.1 | RQ1: Detection Accuracy | 44 |
| 5.1.2 | RQ2: Explainability | 45 |
| 5.1.3 | RQ3: Robustness | 46 |
| 5.2 | Practical Implementation Considerations | 47 |
| 5.2.1 | Window Size and Threshold Selection | 47 |
| 5.2.2 | Drift Type Interpretation | 47 |
| 5.2.3 | Proposed MLOps Deployment Architecture | 48 |
| 5.3 | MLOps Prescription Framework | 49 |
| 5.4 | Limitations and Threats to Validity | 50 |
| 5.5 | Chapter Summary | 51 |

# CONTENTS (cont.)


| | Page |
|---|---|
| **CHAPTER VI CONCLUSION** | **52** |
| 6.1 Summary of Contributions | 52 |
| 6.2 Implications for MLOps | 53 |
| 6.3 Limitations | 54 |
| 6.4 Future Work | 55 |
| 6.5 Concluding Remarks | 56 |
| **APPENDIX** | **58** |
| **APPENDIX A IMPLEMENTATION DETAILS** | **59** |
| A.1 Adversarial Classifier (LightGBM) | 59 |
| A.2 Windowing Strategy | 59 |
| A.3 Permutation Test | 60 |
| A.4 SHAP Configuration | 60 |
| A.5 Computational Environment | 60 |
| **BIOGRAPHY** | **67** |

# LIST OF TABLES


| Table | | Page |
|---|---|---|
| 2.1 | Comparison of Drift Detection Approaches | 10 |
| 3.1 | Alignment of EADD Components with Research Requirements | 22 |
| 4.1 | Experiment 1: Detection Performance Across Temporal Drift Patterns (mean over 5 runs) | 37 |
| 4.2 | Experiment 2: Benchmark Results on Datasets with Known Drift Points | 38 |
| 4.3 | Experiment 2: Detection Activity on Datasets Without Ground Truth | 38 |
| 4.4 | Experiment 3: SHAP Feature Attribution Accuracy | 39 |
| 4.5 | Experiment 4: Mean False Alarm Counts on Stable Data (Zero Actual Drift, 5 runs) | 41 |
| 4.6 | Summary of Experimental Results | 43 |
| 4.7 | Summary of Hypothesis Testing | 43 |
| 5.1 | Guidelines for Window Size Selection Based on Experimental Evidence | 47 |
| 5.2 | MLOps Prescription Framework Based on Drift Diagnosis | 49 |
| A.1 | LightGBM Hyperparameters | 59 |

# LIST OF FIGURES

**Figure** **Page**

2.1 Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources (AUC $\approx$ 0.5). Right: When drift occurs, distributions shift and the classifier can distinguish sources (AUC $>>$ 0.5). 5

3.1 Overview of Explainable Adversarial Drift Detection (EADD) methodology. Blue boxes indicate novel contributions: Permutation Test (Step 3) and SHAP Feature Attribution (Step 4). The framework extends D3's adversarial validation with statistical rigor and root cause analysis. 20

# CHAPTER I
# INTRODUCTION

## 1.1 Motivation

Machine learning models deployed in real-world environments frequently encounter evolving data distributions over time. According to Lu et al. (2018) and Gama et al. (2014), these changes—collectively referred to as distribution shift or drift—can manifest as changes in input features (covariate drift) or in the relationship between inputs and labels (concept drift), causing model performance to deteriorate if left unmanaged. Lu et al. (2018) and Widmer and Kubat (1996) emphasize that detecting such changes early is essential for maintaining reliable model performance, reducing operational risk, and avoiding the accumulation of technical debt in production systems (Bayram et al., 2022; Sculley et al., 2015).

Conventional model monitoring strategies react to drift only after observable performance degradation, which by design means that the system has already suffered from poor predictions on new data (Widmer and Kubat, 1996). In production contexts where ground-truth labels may be delayed, unavailable, or costly to obtain, this post-hoc reactive approach is often inadequate (Lu et al., 2018; Souza et al., 2020a). According to Lu et al. (2018) and Bayram et al. (2022), early drift detection that anticipates distribution changes before significant performance loss is therefore a practical priority in dynamic machine learning systems.

The consequences of undetected covariate drift are severe. A recent comprehensive study of 127 deep-learning models deployed across 47 hospitals demonstrated that demographic shifts in patient populations resulted in a 23.4% decrease in diagnostic accuracy over just eight months (Mannapur, 2025). In critical care settings, undetected drift in patient risk assessment models led to a 34% increase in false negatives. In these high-stakes environments, a simple “drift detected” alarm is insufficient; clinicians and

engineers require immediate root-cause explanations to safely recalibrate models without disrupting patient care.

Current drift detection methods vary considerably in their assumptions and approach. Traditional statistical methods employ hypothesis tests—such as the Kolmogorov–Smirnov test, chi-squared test, or Kullback–Leibler divergence—to detect distribution changes by comparing incoming and reference data (Rabanser et al., 2019; Raab et al., 2020). Bifet and Gavaldà (2007) developed adaptive windowing methods like ADWIN that dynamically adjust how much historical data is retained for comparison. Supervised error-based detectors like DDM and EDDM monitor model performance metrics over time, but require access to ground-truth labels (Gama et al., 2004; Baena-García et al., 2006). Prediction-based detectors track model outputs for changes in confidence, entropy, or prediction distributions without necessarily requiring labels (Sethi and Kantardzic, 2017). Despite this diversity, several challenges remain: statistical tests can be sensitive to noise and may struggle in high-dimensional settings; supervised methods depend on label availability; prediction-based methods may be confounded by model calibration issues; and hybrid approaches often require engineered context variables that may not generalize across domains (Lu et al., 2018; Webb et al., 2016).

To address these gaps, we propose recasting feature drift detection as a discriminative learning problem: treating the comparison between historical and newly arriving data as a binary classification task. If an auxiliary classifier can reliably separate data from the two time periods based solely on features, it indicates a meaningful distributional change. This approach, known as adversarial validation, has been used in data science to detect and quantify dataset shift Pan et al. (2020); Qian and Rao (2021) but has not been systematically developed into a general feature drift detector for streaming or window-based data monitoring.

According to Lopez-Paz and Oquab (2017), this approach offers several key advantages for drift detection: it is label-independent (does not require ground-truth labels for new data), model-agnostic (can be applied regardless of the primary prediction model), makes no assumptions about which features or contexts are important, and can detect complex high-dimensional, non-linear distributional shifts that may evade univariate or marginal statistical tests.

## 1.2 Objectives

The primary objective of this thesis is to develop **Explainable Adversarial Drift Detection (EADD)**, a novel framework that extends adversarial validation with root cause analysis and actionable prescriptions. Specifically, this work aims to:

1. Formalize feature drift detection as a discriminative two-sample learning problem, extending beyond binary detection to provide **feature-level attribution** of drift sources using SHAP-based interpretability methods (Lundberg and Lee, 2017).
2. Develop streaming and sliding window procedures with **reservoir sampling** for representative reference windows, and **permutation testing** for statistically principled threshold calibration.
3. Systematically benchmark EADD against state-of-the-art unsupervised drift detectors—including D3 Gözüaçık et al. (2019), ADWIN Bifet and Gavaldà (2007), KSWIN Raab et al. (2020), and MMD Gretton et al. (2012)—on the real-world datasets from the Lukats et al. (2025) benchmark.
4. Design and validate an **automated prescription system** that provides actionable remediation recommendations based on drift diagnosis (univariate drift, multivariate concept shift, or feature-subset drift).
5. Evaluate practical applicability in MLOps settings, including computational efficiency, threshold sensitivity, and integration with model retraining strategies.

## 1.3 Expected Contributions

This thesis seeks to contribute:

1. **EADD Framework:** A formal explainable adversarial validation framework that combines discriminative drift detection with SHAP-based root

cause analysis, transforming drift detection from a binary alarm into an intelligent diagnostic tool.

2. **Root Cause Analysis:** SHAP-based feature importance extraction that identifies which specific features drive detected drift, addressing the "black box" limitation of existing methods like D3 (Gözüaçık et al., 2019).
3. **Automated Prescriptions:** A novel classification of drift patterns (univariate, multivariate, feature-subset) with corresponding actionable remediation recommendations for MLOps teams.
4. **Benchmark Evaluation:** Comprehensive comparison with state-of-the-art unsupervised detectors on real-world data streams, demonstrating both detection accuracy and the value of explainability.
5. **Practical Guidelines:** Deployment recommendations for MLOps pipelines, including permutation-based threshold calibration, reservoir sampling for reference windows, and computational efficiency considerations.

# CHAPTER II
# LITERATURE REVIEW

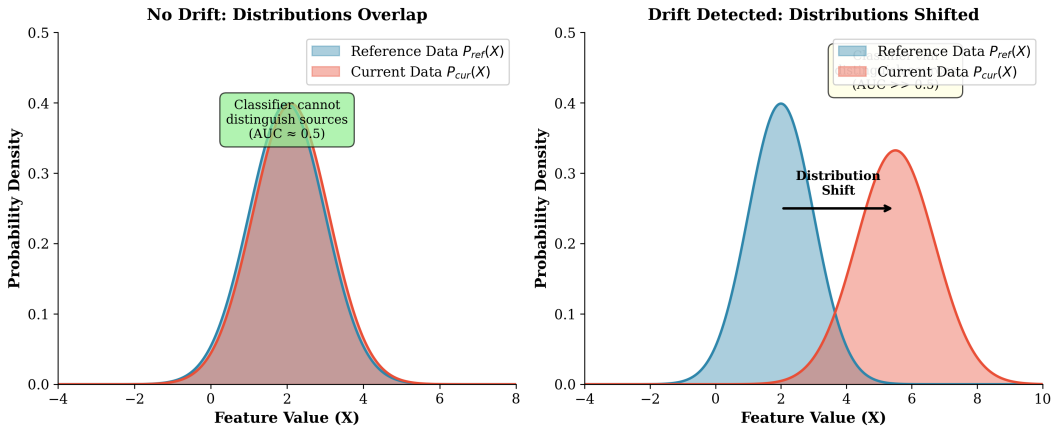


Figure 2.1: Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources (AUC $\approx$ 0.5). Right: When drift occurs, distributions shift and the classifier can distinguish sources (AUC $>>$ 0.5).

## 2.1 Overview

Drift detection addresses the challenge that arises when the statistical characteristics of data evolve over time in a manner that impacts the validity of a machine learning model (Lu et al., 2018; Gama et al., 2014). According to Bayram et al. (2022), when users, environments, data sources, or the relationships between labels and features change, model accuracy can diminish. Gama et al. (2014) and Sculley et al. (2015) emphasize that drift detection identifies such modifications early, allowing practitioners to adapt, retrain, or update the model accordingly. A robust understanding of drift types and detection methods is essential for designing effective monitoring systems.

## 2.2 Types of Drift

Drift can be categorized along two main dimensions: by the nature of the change (what component of the data-generating process is affected) and by the temporal pattern of the change (how the drift manifests over time) (Lu et al., 2018; Gama et al., 2014).

### 2.2.1 Drift by Nature of Change

Based on which component of the joint distribution $P(X, y)$ changes, the literature (Lu et al., 2018; Gama et al., 2014; Tsymbal, 2004) classifies drift into the following types:

**Concept Drift (Real Drift):** The conditional distribution $P(y|X)$ changes over time while the feature distribution $P(X)$ may or may not change (Lu et al., 2018; Žliobaitė, 2010). This represents a change in the underlying relationship between inputs and outputs—the "concept" the model has learned becomes invalid. For example, the factors that determine creditworthiness may evolve due to economic shifts, even if the applicant characteristics (features) remain similar (Gama et al., 2014). Concept drift directly degrades model performance because the learned decision boundaries no longer reflect the true input-output relationship (Lu et al., 2018).

**Covariate Drift (Feature Drift / Virtual Drift):** The marginal distribution of input features $P(X)$ changes over time while the conditional relationship $P(y|X)$ remains stable (Tsymbal, 2004; Shimodaira, 2000). The ground-truth concept does not change, but the model may encounter inputs outside its training distribution. For example, a fraud detection model trained on transaction patterns from one demographic may receive data from a different user population, even though the definition of fraud remains unchanged (Tsymbal, 2004). Covariate drift can degrade model performance because the model is exposed to feature regions where it may not generalize well (Shimodaira, 2000).

**Prior Probability Drift (Label Shift):** The marginal distribution of labels $P(y)$ changes over time while $P(X|y)$ remains stable (Tsymbal, 2004). This occurs when the prevalence of different classes changes—for instance, a seasonal increase in a particular disease affecting the class balance in medical diagnosis. This is sometimes called class imbalance shift (Lu et al., 2018).

**Note on Terminology:** The literature uses varied terminology (Lu et al., 2018). The term "concept drift" is sometimes used as an umbrella term for any distribution change affecting model performance (Gama et al., 2014), while some reserve it specifically for changes in $P(y|X)$ (Tsymbal, 2004). This thesis focuses primarily on covariate (feature) drift detection, as it enables early warning without requiring labels.

### 2.2.2 Drift by Temporal Pattern

Based on how drift manifests over time, the literature (Lu et al., 2018; Gama et al., 2014; Žliobaitė, 2010) categorizes drift as follows:

- **Abrupt (Sudden) Drift:** A rapid, discrete change where the data distribution shifts quickly from one state to another within a short time period (Gama et al., 2014).
- **Gradual Drift:** A new distribution progressively replacing the old one over an extended duration, with samples from both distributions interleaved during the transition period (Lu et al., 2018).
- **Incremental Drift:** Small but continuous changes accumulating over time, eventually resulting in a significant distributional shift (Žliobaitė, 2010).
- **Recurring (Seasonal) Drift:** Previously observed distributions reappearing periodically due to cyclical patterns, such as seasonal effects in retail or recurring user behavior patterns (Lu et al., 2018; Gama et al., 2014).

Understanding drift patterns is important for detector design: abrupt drift requires rapid response, gradual drift requires sustained sensitivity, and recurring drift benefits from mechanisms that can recognize previously seen distributions (Gama et al., 2014).

## 2.3 Classification of Drift Detection Methods

Based on comprehensive surveys of drift detection literature (Lu et al., 2018; Gama et al., 2014; Tsymbal, 2004; Hinder et al., 2024), drift detection methods can be

classified by multiple dimensions. According to Lu et al. (2018), the most fundamental distinction is based on label availability: whether the method requires access to ground-truth labels (supervised) or operates without them (unsupervised).

### 2.3.1 Supervised Drift Detection Methods

According to Gama et al. (2014) and Gama et al. (2004), supervised drift detection methods require access to ground-truth labels and primarily monitor model performance or prediction errors to detect distributional changes. Gama et al. (2004) and Baena-García et al. (2006) explain that these methods operate under the assumption that unexpected changes in error characteristics—such as increases in error rate, loss magnitude, variance, or cumulative deviation—are indicative of underlying drift.

Key supervised drift detection algorithms include:

**Drift Detection Method (DDM):** According to Gama et al. (2004), DDM monitors the online error rate and its standard deviation. When these statistics exceed predefined thresholds derived from binomial distribution bounds, DDM signals a warning or drift state. Gama et al. (2004) note that DDM is effective for abrupt drift but may be slow to detect gradual changes.

**Early Drift Detection Method (EDDM):** EDDM extends DDM by tracking the distance between consecutive classification errors rather than just the error rate (Baena-García et al., 2006). This modification improves sensitivity to gradual drift by detecting when errors cluster more closely together.

**Page–Hinkley Test:** This sequential analysis technique uses cumulative sums to detect changes in the mean of a monitored signal (Page, 1954). When cumulative deviation exceeds a threshold, drift is indicated.

**Hoeffding Drift Detection Methods (HDDM-A, HDDM-W):** These methods use Hoeffding's inequality to provide statistically grounded drift decisions with theoretical guarantees (Frias-Blanco et al., 2015). HDDM-A detects abrupt changes using moving averages, while HDDM-W is designed for gradual drift using window-based statistics.

**Limitations:** Supervised methods fundamentally require labeled data, which may be delayed, expensive, or unavailable in many production scenarios (Lu et al.,

2018; Sethi and Kantardzic, 2017). They also react to drift after it has already caused performance degradation, rather than detecting distributional change proactively.

### 2.3.2 Unsupervised Drift Detection Methods

Unsupervised drift detection methods do not require ground-truth labels and instead monitor changes in data distribution, model outputs, or internal representations (Tsymbal, 2004; Hinder et al., 2024). These methods enable earlier detection of distributional shifts—before performance degradation becomes observable—making them particularly valuable in production environments with label delays (Sethi and Kantardzic, 2017).

#### 2.3.2.1 Statistical Distribution-Based Methods

Statistical test–based methods detect drift by directly examining changes in data distributions through hypothesis testing or distance-based measures (Rabanser et al., 2019; Raab et al., 2020). A common strategy compares a reference data window against a current data window to test whether they originate from the same distribution (Bifet and Gavaldà, 2007).

Key statistical drift detection methods include:

**Adaptive Windowing (ADWIN):** ADWIN dynamically maintains a sliding window and automatically shrinks it when statistically significant distributional change is detected (Bifet and Gavaldà, 2007). It provides theoretical guarantees on false positive and false negative rates and adapts its window size based on observed change.

**KSWIN:** This method applies the Kolmogorov–Smirnov two-sample test over sliding windows to detect changes in one-dimensional feature distributions (Raab et al., 2020). KSWIN is non-parametric and makes minimal assumptions about the data distribution.

**Chi-Squared Test-Based Detectors:** These use the chi-squared test to compare categorical feature distributions or discretized continuous features between time periods (Lu et al., 2018).

**Kullback–Leibler (KL) Divergence and Other Divergence Measures:** Methods that quantify the "distance" between probability distributions, including KL divergence, Jensen–Shannon divergence, and Hellinger distance, can all be used to

measure distributional shift (Dasu et al., 2006).

**Maximum Mean Discrepancy (MMD):** MMD is a kernel-based two-sample test that compares distributions by mapping them into a reproducing kernel Hilbert space, capable of capturing complex multivariate distributional differences (Gretton et al., 2012).

**Strengths and Limitations:** Statistical tests are interpretable, theoretically grounded, and label-free (Rabanser et al., 2019). However, they can be sensitive to noise (causing false alarms), may struggle in high-dimensional spaces due to the curse of dimensionality, and typically test marginal distributions rather than joint distributions (Lu et al., 2018; Webb et al., 2016).

**Comparison of Statistical Methods with Adversarial Validation:**

Table 2.1 compares popular univariate statistical methods (PSI, KS) with the multivariate Adversarial Validation approach used in this thesis.

Table 2.1: Comparison of Drift Detection Approaches

| Method | Approach | Strengths | Limitations |
|---|---|---|---|
| **PSI (Population Stability Index)** | Compares binned distributions of single features; PSI $> 0.25$ indicates significant drift | Simple, interpretable, industry standard | Univariate only; requires binning; misses feature correlations |
| **KS Test (Kolmogorov-Smirnov)** | Non-parametric test comparing cumulative distributions per feature | No distributional assumptions; exact p-values | Univariate only; multiple testing problem with many features |
| **Adversarial Validation** | Trains classifier to distinguish old vs. new data; uses AUC as drift signal | Multivariate; captures feature interactions; single unified test | Requires classifier training; no built-in explainability (addressed by EADD) |

The key distinction is that PSI and KS are **univariate** methods—they test each feature independently. When monitoring $d$ features, this requires $d$ separate tests, leading to the "false alarm storm" problem where the probability of at least one false positive increases with $d$. Adversarial Validation is inherently **multivariate**, testing the joint distribution $P(X_1, X_2, \ldots, X_d)$ in a single unified check.

#### 2.3.2.2 Model-Based Discriminative Methods

According to Pan et al. (2020), Lopez-Paz and Oquab (2017), and Sethi and Kantardzic (2017), discriminative drift detection methods train an auxiliary model to distinguish between reference and current data distributions. Lopez-Paz and Oquab (2017) explain that the core idea is to frame drift detection as a classification problem: if a classifier can accurately predict whether each instance originates from the reference or current distribution, the distributions must be distinguishably different.

Key approaches include:

**Domain Classifier / Adversarial Validation:** According to Pan et al. (2020) and Qian and Rao (2021), this approach trains a binary classifier (e.g., logistic regression, random forest, gradient boosting) to discriminate samples from reference vs. current windows. Pan et al. (2020) note that high discriminative performance (measured by accuracy, AUC, or similar metrics) indicates significant distributional shift.

**Discriminative Drift Detector (D3):** Gözüaçık et al. (2019) developed this unsupervised method that trains a classifier (logistic regression) to distinguish between old and new data windows, using AUC as the drift signal. D3 has demonstrated strong performance in recent benchmarks (Lukats et al., 2025) and represents the state-of-the-art in discriminative drift detection.

**Margin Density Drift Detection (MD3):** Sethi and Kantardzic (2017) proposed an alternative approach that monitors margin density—the proportion of samples near the decision boundary—to detect drift without requiring true labels.

**Margin-Based and Density-Based Discriminators:** According to Ditzler and Polikar (2011), these methods monitor changes in classifier margin densities or decision boundary proximity to detect distributional shifts.

**Strengths:** Discriminative methods can capture high-dimensional, non-linear distributional shifts that marginal univariate tests may miss (Lopez-Paz and Oquab, 2017). They require no labels for the drift detection task itself (only pseudo-labels indicating data source) and naturally aggregate information across all features (Pan et al., 2020).

**Limitations:** These methods require training a classifier at each monitoring step, which has computational cost; performance depends on classifier choice and hyperparameters; and interpretation of drift signals may be less intuitive than statistical test

p-values (Sethi and Kantardzic, 2017).

#### 2.3.2.3 Representation-Based Methods

Representation-based methods detect drift by monitoring changes in learned representations or embeddings rather than raw feature distributions (Rabanser et al., 2019; Liu et al., 2020):

**Embedding Distribution Shift:** Computing statistical distances (cosine similarity, Wasserstein distance, MMD) between reference and current embedding distributions derived from neural network intermediate layers (Rabanser et al., 2019).

**Deep Learning Representation Monitoring:** This approach tracks stability of learned representations across time, leveraging insights from domain adaptation and transfer learning literature (Long et al., 2016).

These methods are particularly relevant for deep learning systems where raw feature distributions may not reflect semantically meaningful changes (Liu et al., 2020).

#### 2.3.2.4 Prediction-Based / Output-Based Methods

Prediction-based methods detect drift by monitoring changes in model outputs over time without requiring evaluation of prediction correctness (Sethi and Kantardzic, 2017; Gözüaçık et al., 2019):

**Prediction Distribution Shift:** Monitoring changes in the distribution of model predictions (class probabilities, regression outputs) over time (Gözüaçık et al., 2019).

**Uncertainty / Entropy Monitoring:** Tracking prediction entropy or uncertainty scores under the assumption that rising uncertainty reflects exposure to out-of-distribution data (Guo et al., 2017).

**Confidence Monitoring:** Tracking changes in model confidence levels and softmax output distributions (Guo et al., 2017).

**Limitations:** These methods depend on model calibration. If the model is poorly calibrated—which is common for modern deep learning models—changes in confidence may not reliably indicate drift (Guo et al., 2017; Ovadia et al., 2019).

#### 2.3.2.5 State-of-the-Art Unsupervised Detectors (2019–2025)

Recent benchmarking by Lukats et al. (2025) provides a systematic evaluation of fully unsupervised drift detectors on real-world data streams. Their study tested

numerous algorithms and identified seven that function reliably on messy, real-world data. This survey represents a critical "reality check" for the field, demonstrating that detecting change without ground-truth labels is possible but fragile.

**D3 (Discriminative Drift Detector):** According to Gözüaçık et al. (2019), D3 trains a logistic regression classifier to distinguish between old and new data windows, using AUC as the drift signal. Lukats et al. (2025) found D3 to be the overall winner—the most robust and accurate across different data types. However, D3 remains a "black box": it signals *that* drift occurred but provides no insight into *which* features drifted or *why*.

**CSDDM (Clustered Statistical Drift Detection Method):** According to Hsieh et al. (2021), this method uses PCA for dimensionality reduction followed by K-Means clustering, then applies statistical tests to detect cluster shifts. Lukats et al. (2025) report it performed exceptionally well on the "lift-per-drift" metric, catching drifts with minimal false alarms.

**SPLL (Semi-Parametric Log-Likelihood):** Kuncheva (2013) developed this approach that fits probability density models to reference data and detects drift when new observations have anomalously low likelihood under the learned distribution. It is effective for sudden drift patterns.

**IBDD (Image-Based Drift Detector):** According to Souza et al. (2020b), this creative approach converts numerical data streams into images and applies image comparison techniques to detect distributional changes. While highly accurate, Lukats et al. (2025) note it can be biased by its frequent reset strategy.

**OCDD (One-Class Drift Detector):** Gözüaçık and Can (2021) proposed using one-class SVMs to define "normal" data boundaries and flag drift when incoming data falls outside these boundaries. While sound in principle, D3 generally outperformed OCDD in benchmarks (Lukats et al., 2025).

**NN-DVI (Nearest Neighbor Density Variation):** According to Liu et al. (2018), this method monitors changes in local data density by tracking nearest-neighbor distances. Drift is signaled when density patterns shift significantly. It performs well for gradual density changes but can be sensitive to noise.

**UCDD (Unsupervised Concept Drift Detection):** Shang et al. (2020) developed a clustering-based approach that assigns pseudo-labels based on cluster membership

and monitors boundary shifts. It serves as a reliable baseline in unsupervised drift detection.

**BNDM (Bayesian Non-parametric Drift Monitor):** This method uses a Pólya tree test to detect distributional changes without parametric assumptions. However, Lukats et al. (2025) report that BNDM struggled with small sample sizes in their benchmark, limiting its practical applicability in streaming scenarios where window sizes are constrained.

**UDetect (Univariate Detection):** This approach monitors standard deviation changes using Shewhart control charts. While simple and computationally efficient, Lukats et al. (2025) found it had a high failure rate (45%) in their benchmark, making it unreliable for production deployment.

**Industrial Drift Detection Libraries:** Beyond research algorithms, several industrial libraries provide production-ready drift detection:

- **Alibi Detect**: Python library implementing Kolmogorov-Smirnov tests, MMD, and learned kernel methods for multivariate drift detection (Liu et al., 2020).
- **NannyML**: Confidence-based Performance Estimation (CBPE) for monitoring model performance without ground-truth labels (Bayram et al., 2022).

**Gap Identified from Benchmark Analysis:** While these libraries and algorithms exist, according to Lukats et al. (2025), they share a common limitation: they typically report only *whether* drift occurred, not *where* or *why*. Statistical libraries often run separate univariate tests for each feature, causing "false alarm storms" when many features are monitored simultaneously. The field lacks methods that provide a unified multivariate check with granular explanations of drift sources.

### 2.3.3 Hybrid and Ensemble Methods

Hybrid approaches integrate multiple sources of information—both supervised and unsupervised signals—to improve drift detection robustness (Gama et al., 2014; Krawczyk et al., 2017):

**Context-Aware Drift Detection:** Combining model predictions, user context, operational signals, and metadata to compute context-weighted drift scores (Krawczyk et al., 2017).

**Multi-Signal Ensemble Methods:** Integrating predictions, embeddings, operational KPIs, and metadata into a unified drift score (Krawczyk et al., 2017).

**Drift Detection Ensembles:** Combining multiple base detectors using voting, weighting, or meta-learning to reduce both false alarms and missed detections (Krawczyk et al., 2017).

## 2.4 Limitations of Existing Methods

While the drift detection literature offers diverse approaches, each category has characteristic limitations:

**Supervised Methods (DDM, EDDM, HDDM):** These require timely labeled data, which is often delayed or unavailable in production (Lu et al., 2018). They detect drift reactively—only after performance degradation has already occurred (Sethi and Kantardzic, 2017).

**Statistical Distribution Methods (ADWIN, KSWIN, KL-Divergence):** These can be sensitive to noise, leading to false alarms (Rabanser et al., 2019). They may struggle in high-dimensional or correlated feature spaces and typically test marginal distributions, potentially missing joint distribution changes (Lu et al., 2018; Webb et al., 2016).

**Prediction/Uncertainty-Based Methods:** These rely on model calibration. Modern ML models are often poorly calibrated, especially under distribution shift, causing these signals to be unreliable (Guo et al., 2017; Ovadia et al., 2019).

**Context-Aware Methods:** These require pre-specification of relevant context variables, which may be incomplete, noisy, or themselves drift over time (Krawczyk et al., 2017). Misspecified context can cause missed detections or spurious alarms.

**Fundamental Gap:** Critically, most existing methods do not directly test whether new data is statistically distinguishable from historical data in the original feature

space (Lopez-Paz and Oquab, 2017). Discriminative approaches offer a natural, model-agnostic signal of drift that does not depend on predefined contexts, labels, or calibrated uncertainties—yet this perspective has received limited systematic attention in the drift detection literature (Pan et al., 2020).

This thesis aims to bridge this gap by establishing adversarial validation as a viable, general-purpose drift detection technique through rigorous empirical evaluation and practical methodology development.

## 2.5 Related Work: Adversarial Validation

### 2.5.1 Concept and Origins

According to Pan et al. (2020), adversarial validation was originally proposed as a practical technique for diagnosing dataset shift between training and evaluation data. Pan et al. (2020) and Qian and Rao (2021) explain that the approach reframes the comparison of two datasets as a binary classification problem: train a classifier to predict whether each instance originates from the reference distribution (e.g., training data) or the target distribution (e.g., test or new data). Lopez-Paz and Oquab (2017) note that high discriminative performance indicates statistical separability—hence significant distributional shift—while near-random performance suggests the datasets are largely indistinguishable.

According to Pan et al. (2020), the term "adversarial" reflects the insight that if an adversary (the classifier) cannot distinguish the data sources, the distributions must be similar. Lopez-Paz and Oquab (2017) note this is related to the discriminator component in Generative Adversarial Networks (GANs), though applied here for diagnostic rather than generative purposes.

### 2.5.2 Industrial Applications

**Uber's MaLTA System:** Pan et al. (2020) presented adversarial validation at Uber for their MaLTA (Machine Learning Targeting Automation) user targeting system. They trained a classifier to distinguish historical training data from newly observed targeting data. When the adversarial classifier exhibited strong separability, it signaled

train-deployment mismatch, prompting feature filtering and retraining adjustments before deployment. Their results demonstrated improved robustness compared to naive retraining strategies that react only after observable performance degradation.

**Credit Scoring:** Qian and Rao (2021) applied adversarial validation to manage dataset shift in credit scoring. They trained an adversarial classifier to distinguish historical credit applications from recent ones, using the classifier's predictions to filter training samples—keeping only those most similar to the current deployment distribution for model training.

**Image Classification and Operations:** Additional applications include using adversarial validation to quantify dataset complexity in image classification Renza and Moya-Albor (2024) and to detect when historical data is no longer representative in manufacturing/operations settings Senoner et al. (2023).

**Feature Attribution Monitoring at Google:** Taly and Sato (2021) described how Google's Vertex AI platform uses feature attribution monitoring to detect model degradation in production. Their system computes feature attribution baselines during model training and continuously compares production attributions against these baselines. When attributions diverge significantly, the system alerts engineers with diagnostic information about which features changed—enabling targeted debugging rather than blind retraining. Notably, this approach detected a critical service degradation incident that traditional input-distribution monitoring missed, because the drift manifested in the model's internal feature importance rather than in raw input statistics. This industrial deployment at Google scale validates the core principle underlying EADD: that feature-level attribution analysis provides superior drift diagnostics compared to aggregate distribution tests.

**Cloud Industry Standardization:** The necessity of feature attribution monitoring is rapidly becoming an industry standard across major cloud providers. As with Google's Vertex AI, Amazon Web Services (AWS) has integrated "Feature Attribution Drift" monitoring into Amazon SageMaker Clarify (Amazon Web Services, 2025). SageMaker automatically calculates a SHAP baseline during training and actively monitors production data for deviations in these attribution scores. This cross-platform adoption by leading cloud providers validates the core premise of EADD: MLOps drift detection is

increasingly moving toward continuous, explainable feature attribution rather than basic distribution tracking.

## 2.6 Gap in the Literature

While discriminative methods like D3 have proven that adversarial validation can accurately detect drift (Gözüaçık et al., 2019; Lukats et al., 2025), significant gaps remain that limit their practical utility in production MLOps environments:

1. **The "Black Box" Problem:** According to Lukats et al. (2025), existing discriminative detectors like D3 signal *that* drift occurred but provide no insight into *which* features drifted or *why*. In complex ML systems, knowing that drift happened is insufficient—engineers need to understand *what drifted* to take appropriate corrective action.
2. **No Root Cause Analysis:** Current methods provide only a binary detection signal without identifying the specific source of the change (Lukats et al., 2025). Production systems require diagnostic capabilities that pinpoint drift sources and guide remediation.
3. **False Alarm Storm Problem:** Industrial libraries often run separate univariate tests for each feature, causing multiple simultaneous alerts when features are correlated (Rabanser et al., 2019). This overwhelms operators rather than guiding them toward the actual drift source.
4. **Threshold Calibration Challenges:** According to Lukats et al. (2025), determining appropriate detection thresholds remains a major unsolved problem. Existing methods either use arbitrary thresholds or require labeled validation data that may not be available.
5. **No Actionable Prescriptions:** Even when drift is detected, current methods provide no guidance on appropriate response strategies—whether to retrain fully, adjust specific features, or apply targeted corrections (Bayram et al., 2022).

According to recent benchmarks (Lukats et al., 2025), D3 represents the state-of-the-art in detection accuracy. However, its lack of explainability severely limits practical deployment. This thesis proposes **Explainable Adversarial Drift Detection (EADD)**, which extends adversarial validation with SHAP-based root cause analysis (Lundberg and Lee, 2017) to transform drift detection from a simple alarm into an intelligent diagnostic tool.

This thesis aims to bridge this explainability gap by developing a framework that not only detects drift with high accuracy but also provides actionable insights about drift sources and appropriate remediation strategies.

# CHAPTER III
# METHODOLOGY

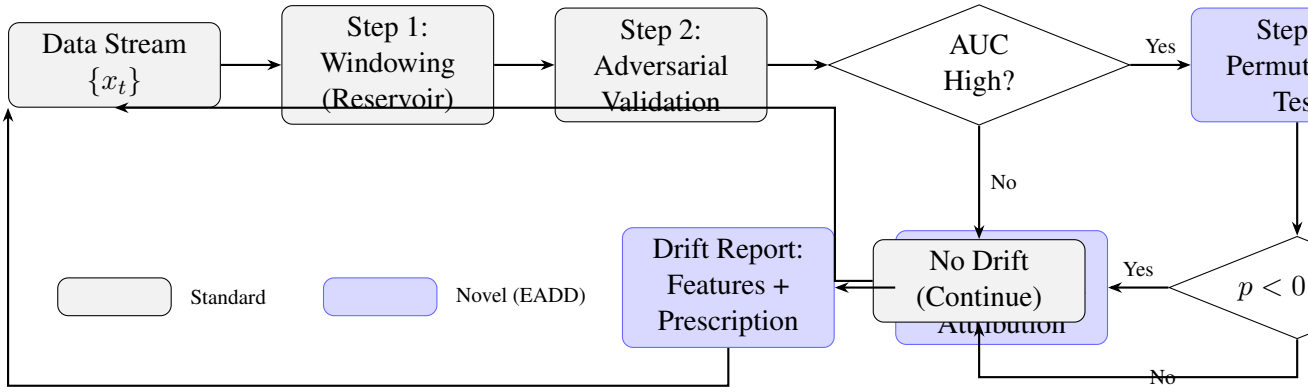


Figure 3.1: Overview of Explainable Adversarial Drift Detection (EADD) methodology. Blue boxes indicate novel contributions: Permutation Test (Step 3) and SHAP Feature Attribution (Step 4). The framework extends D3's adversarial validation with statistical rigor and root cause analysis.

## 3.1 Overview: Explainable Adversarial Drift Detection (EADD)

This thesis proposes **Explainable Adversarial Drift Detection (EADD)**, a framework designed specifically for **Feature Drift** ($P(X)$ shift). The fundamental insight is that **Adversarial Validation** and **Feature Drift Detection** are mathematically equivalent—both ask: *"Can a model distinguish between old data and new data based solely on features?"*

While existing methods like the Discriminative Drift Detector (D3) (Gözüaçık et al., 2019) successfully employ adversarial validation to flag distributional changes, they function as opaque detectors—providing a drift signal without explaining *which* features drifted. Conventional drift detection methods produce only binary alerts; EADD

is designed as a diagnostic framework that identifies exactly which features contributed to the drift, enabling targeted corrective action.

EADD extends the D3 architecture with three key innovations:

1. **Permutation Test:** Statistical validation ensuring detected drift is significant, not noise
2. **SHAP-based Root Cause Analysis:** Granular feature attribution identifying *which* features drifted
3. **Automated Prescriptions:** Actionable remediation recommendations based on drift diagnosis

## 3.2 Problem Formulation

According to Pan et al. (2020) and Lopez-Paz and Oquab (2017), this research formalizes feature (covariate) drift detection as a problem of discriminating between historical reference data and newly arriving observations in a streaming or windowed setting. Shimodaira (2000) explains that the objective is to detect when the underlying feature distribution $P(X)$ changes substantially over time, providing an early indication of distributional drift before observable degradation in task performance occurs.

**Formal Definition:** Let $\{x_t\}_{t=1}^{T}$ denote a stream of feature vectors arriving over time. At each monitoring point $t$, a drift detector produces a binary decision $d_t \in \{\text{no-drift}, \text{drift}\}$ based on comparison between:

- A reference window $W_{ref}$ containing historical samples presumed to represent the stable/baseline distribution
- A current window $W_{cur}$ containing the most recent observations under test

**Extended Objective:** Unlike existing detectors that output only a binary drift decision, EADD additionally produces:

- A ranked list of features contributing to the drift

- Feature importance scores quantifying each feature's contribution
- An automated prescription for appropriate remediation action

According to Lu et al. (2018) and Gama et al. (2014), the detection task must balance:

- **Promptness:** Low detection delay (lag between true drift and detection)
- **Reliability:** Low false positive rate (false alarms when no drift exists)
- **Sensitivity:** High true positive rate (detecting drift when it occurs)
- **Efficiency:** Bounded computational and memory requirements suitable for production
- **Explainability:** Clear attribution of drift to specific features (novel contribution)

## 3.3 EADD Framework Components

The EADD methodology maps directly to the thesis requirements:

Table 3.1: Alignment of EADD Components with Research Requirements

| **Requirement** | **EADD Solution** |
|---|---|
| "Based on Adversarial Validation" | Step 2: Train classifier to distinguish $W_{ref}$ vs $W_{cur}$ |
| "Based on Permutation Test" | Step 3: Shuffle labels $B$=50 times to calculate p-value |
| "Like D3" | Foundation: Same discriminative logic, upgraded classifier (LightGBM) |
| "Focus on Feature Drift" | Step 4: SHAP identifies which $P(X)$ components shifted |

The methodology consists of four integrated steps:

**Step 1: Adaptive Reference Windowing** — Reservoir sampling for representative baseline

**Step 2: Adversarial Validation (Detector)** — Train discriminative classifier on $W_{ref}$ vs $W_{cur}$

**Step 3: Permutation Test (Significance)** — Statistical validation via label shuffling

**Step 4: Feature Drift Explainability (SHAP)** — Identify which features drifted (novel contribution)

### 3.3.1 Step 1: Adaptive Reference Windowing

To detect Feature Drift, we must compare the current data stream against a stable baseline. EADD maintains two windows consistent with established methodologies (Bifet and Gavaldà, 2007; Tsymbal, 2004):

**Reference Window ($W_{ref}$):** A buffer containing $N_{ref}$ historical samples representing the stable feature distribution $P_{ref}(X)$. Unlike fixed sliding windows that "forget" normal data too quickly, EADD uses **Reservoir Sampling** (Vitter, 1985) to ensure the reference captures the global distribution of features, not just the most recent batches.

**Current Window ($W_{cur}$):** A buffer containing the $N_{cur}$ most recent observations representing the current feature distribution $P_{cur}(X)$. As new data arrives, older samples are discarded to maintain responsiveness to recent changes.

**Update Mechanism:** At each monitoring step:

- New samples enter $W_{cur}$
- Oldest samples exit $W_{cur}$
- Reservoir sampling maintains representative $W_{ref}$
- Upon confirmed drift, $W_{ref}$ may be reset to current distribution

According to Lu et al. (2018), window sizes represent a bias-variance tradeoff: larger windows provide more stable estimates but slower response; smaller windows enable faster detection but higher variance.

### 3.3.2 Step 2: Adversarial Validation (The Detector)

This step implements the core **Adversarial Validation** logic that forms the foundation of both D3 (Gözüaçık et al., 2019) and EADD. The insight is that Feature

Drift detection reduces to a binary classification problem: *can a model tell the difference between old and new data?*

**Construct Binary Classification Dataset:**

- Label all samples from $W_{ref}$ as class 0 ("old" / reference)
- Label all samples from $W_{cur}$ as class 1 ("new" / current)
- Combined dataset: $X = W_{ref} \cup W_{cur}$, $y = \{0\}^{N_{ref}} \cup \{1\}^{N_{cur}}$

**Train Adversarial Classifier:** Train a binary classifier $f_\theta$ to predict data source from features alone. EADD uses **LightGBM** (Ke et al., 2017) as the default classifier.

*Why LightGBM over D3's Logistic Regression?* D3 (Gözüaçık et al., 2019) uses logistic regression, which is linear. LightGBM captures **non-linear feature interactions**, enabling detection of complex Feature Drifts where multiple features shift in correlated ways that linear models miss.

**Measure Discriminative Performance:** Evaluate classifier performance using stratified cross-validation:

- **AUC-ROC:** Primary metric—Area under the ROC curve
- AUC $\approx$ 0.5: Classifier cannot distinguish $W_{ref}$ from $W_{cur}$ $\rightarrow$ **No Feature Drift**
- AUC $>>$ 0.5: Classifier separates windows $\rightarrow$ $P_{ref}(X) \neq P_{cur}(X)$ $\rightarrow$ **Feature Drift detected**

**Objective:** If the adversarial classifier achieves high AUC, the feature distributions $P(X)$ have diverged—this is the definition of Feature Drift.

### 3.3.3 Step 3: Statistical Significance via Permutation Test

A raw AUC score (e.g., 0.65) is inconclusive, as it may reflect genuine distributional change or statistical noise. Standard adversarial validation (Pan et al., 2020) relies on fixed thresholds (e.g., AUC $> 0.7$), which lack principled justification. EADD addresses this limitation with a **Permutation Test** (Lopez-Paz and Oquab, 2017).

**The Permutation Test Procedure:**

1. **Generate Null Distribution:** Randomly shuffle the labels (0 and 1) of the combined dataset $B = 50$ times
2. **Retrain:** For each permutation, retrain the adversarial classifier and compute AUC
3. **Build Null:** The $B$ permuted AUCs form the "Null Distribution" of AUC scores where *no drift exists*
4. **Compare:** Compare the *actual* adversarial AUC against this Null Distribution

**Drift Confirmation Rule:**

- Compute empirical p-value: $p = \frac{\#\{\text{AUC}_{perm} \geq \text{AUC}_{actual}\}}{B}$
- Drift is confirmed **only** if $p < 0.01$ (99th percentile)

**Benefit:** This prevents false alarms caused by small sample sizes or random fluctuations—a common failure mode in standard adversarial validation. The permutation test provides statistical guarantees that the detected drift is real, not noise.

#### 3.3.4 Step 4: Feature Drift Explainability via SHAP (Novel Contribution)

Once the Permutation Test confirms drift ($p < 0.01$), the next objective is to determine *which features drifted*. A key limitation of D3 (Gözüaçık et al., 2019) is that it discards the trained classifier after computing AUC. However, the adversarial classifier has already learned to discriminate between the two distributions, encoding precisely *which* features differentiate the reference data from the current data.

**The SHAP Extraction Method:** EADD applies **SHAP (SHapley Additive exPlanations)** (Lundberg and Lee, 2017) to the adversarial classifier:

1. Compute SHAP values using TreeSHAP (efficient for LightGBM)
2. Calculate mean absolute SHAP value per feature: $\text{Importance}_i = \frac{1}{n} \sum_{j=1}^{n} |\text{SHAP}_{ij}|$
3. Normalize to percentages: $\text{Contribution}_i = \frac{\text{Importance}_i}{\sum_k \text{Importance}_k} \times 100\%$
4. Rank features by contribution

**Interpretation—The Core Insight:**

A feature with a high SHAP value is a feature that **heavily helped the model distinguish "new" data from "old" data**. Therefore: **High SHAP = High Feature Drift**.

**Output Format:** The system outputs a ranked list of "Drifting Features" with contribution percentages:

- Example: "Drift Detected (AUC=0.85, $p<0.01$). Primary Driver: **Age** (contribution: 45%), Secondary Driver: **Income** (contribution: 25%), Other: Location (15%), Tenure (10%), Balance (5%)"

This transforms the drift detector from a binary alarm into an **intelligent diagnostic tool** that pinpoints exactly which features require attention.

**Theoretical Justification for Attribution over Distribution:** The choice of SHAP-based feature attribution over traditional distribution-based monitoring is supported by industrial evidence. Taly and Sato (2021) demonstrated at Google's Vertex AI that feature attribution monitoring detected a critical model degradation that input-distribution monitoring missed. This occurs because:

1. **Attribution captures model-relevant drift:** Distribution tests may flag statistically significant but model-irrelevant changes, while attribution methods focus on changes that actually affect model predictions.
2. **Attribution provides actionable diagnostics:** Knowing which features' *importance* changed (rather than which features' distributions shifted) directly maps to remediation actions.
3. **Attribution handles feature interactions:** SHAP values account for feature interactions through the Shapley value framework, capturing multivariate drift patterns that univariate tests miss.

### 3.3.5 Automated Drift Prescription (Novel Contribution)

Beyond diagnosis, EADD provides actionable remediation recommendations based on the drift pattern identified:

**Scenario A: Univariate Drift (Single Feature Dominant)**

*Condition:* One feature contributes >50% of total importance.

*Prescription:* "Univariate drift detected on [Feature]. Recommended actions:

- Investigate data pipeline for [Feature] (sensor failure, data source change)
- Consider feature-specific preprocessing adjustment
- If persistent: Retrain with reduced weight on [Feature] or exclude temporarily"

**Scenario B: Multivariate Concept Shift (Distributed Importance)**

*Condition:* No single feature exceeds 30% importance; drift is spread across multiple features.

*Prescription:* "Multivariate concept shift detected. The entire data structure has changed. Recommended actions:

- Full model retraining required immediately
- Consider expanding training data to include recent distribution
- Review deployment assumptions and feature engineering"

**Scenario C: Feature Subset Drift**

*Condition:* 2-5 features together contribute >70% of importance.

*Prescription:* "Correlated feature drift detected in [Feature Group]. Recommended actions:

- Investigate common data source for affected features
- Consider partial retraining with emphasis on affected feature subspace
- Evaluate feature engineering for the affected domain"

### 3.3.6 Model Adaptation Strategies

According to Gama et al. (2014), upon drift detection, the primary prediction model can be adapted:

**Strategy 1: Drift-Triggered Retraining**

- Model is trained at initialization
- Retraining occurs only when drift alarm is raised
- Reference window resets to current data after retraining
- According to Pan et al. (2020), advantage: Conserves resources; exploits early detection for timely updates

**Strategy 2: Periodic Retraining (Baseline)**

- Model is retrained at fixed intervals regardless of drift signals
- Advantage: Simple; does not depend on detector reliability
- According to Gama et al. (2014), disadvantage: May retrain unnecessarily or too late

**Strategy 3: Continuous Update with Drift Weighting**

- Online learning with sample weighting based on drift score
- According to Krawczyk et al. (2017), recent samples receive higher weights when drift is detected

## 3.4 Datasets

Following the comprehensive benchmark by Lukats et al. (2025), experiments used both synthetic and real-world data streams to evaluate EADD under controlled and realistic conditions.

**Synthetic Datasets:** Custom synthetic data generators were used to create controlled $P(X)$ shifts with known ground-truth drift points, enabling precise measurement of detection delay and accuracy:

- **SineClusters:** Sine-modulated cluster data with controlled drift patterns
- **WaveformDrift2:** Waveform generator with gradual feature drift
- **Custom Generators:** Streams of $n = 10{,}000$ samples with $d = 5$ features and controlled abrupt, gradual, incremental, and recurring drift injected at known positions (e.g., $t = 5{,}000$)

**Real-World Benchmark Datasets:** Thirteen real-world data streams from the Lukats et al. (2025) benchmark were used, spanning diverse domains and drift characteristics:

- **Electricity (Elec2)** (Harries, 1999): 45,312 samples $\times$ 8 features; electricity pricing data with natural market fluctuations
- **INSECTS Variants** (Souza et al., 2020a): Six variants—Abrupt Balanced, Gradual Balanced, Incremental Balanced, Incremental-Abrupt Balanced, Incremental-Reoccurring Balanced—each containing $\sim$24,000–80,000 samples $\times$ 33 features with documented ground-truth drift points
- **NOAA Weather:** Weather measurements with temporal drift patterns
- **Outdoor Objects:** Object recognition features with environmental variation
- **Ozone:** Atmospheric ozone level data with seasonal drift
- **Poker Hand:** Card game data with complex feature relationships
- **Powersupply:** Power consumption data with usage pattern changes
- **Rialto Bridge Timelapse:** Image-derived features with temporal variation
- **Luxembourg:** Financial data with market-driven distributional shifts

All datasets were processed in streaming fashion, with features used for drift detection. For datasets with known drift points (e.g., INSECTS variants with drift indices at specific sample positions), ground truth was used for computing detection metrics. The INSECTS datasets were particularly valuable as they provide exact drift timestamps, enabling precise measurement of detection delay and false alarm rates.

## 3.5 Baseline Drift Detectors

EADD was benchmarked against seven state-of-the-art unsupervised drift detectors evaluated in the Lukats et al. (2025) survey, all of which operate without ground-truth labels:

- **D3 (Discriminative Drift Detector)** Gözüaçık et al. (2019): Trains a logistic regression classifier to distinguish old from new data windows, using AUC as the drift signal. Identified as the overall best-performing unsupervised detector in the Lukats et al. (2025) benchmark.
- **BNDM (Bayesian Non-parametric Drift Monitor):** Uses a Pólya tree test to detect distributional changes without parametric assumptions.
- **CSDDM (Clustered Statistical Drift Detection Method)** Hsieh et al. (2021): Applies PCA dimensionality reduction followed by K-Means clustering, then uses statistical tests to detect cluster shifts.
- **IBDD (Image-Based Drift Detector)** Souza et al. (2020b): Converts numerical data streams into images and applies image comparison techniques to detect distributional changes.
- **OCDD (One-Class Drift Detector)** Gözüaçık and Can (2021): Uses one-class SVMs to define "normal" data boundaries and flags drift when incoming data falls outside these boundaries.
- **SPLL (Semi-Parametric Log-Likelihood)** Kuncheva (2013): Fits probability density models to reference data and detects drift when new observations have anomalously low likelihood.
- **UDetect (Univariate Detection):** Monitors standard deviation changes using Shewhart control charts.

Existing benchmark results for these seven detectors from the Lukats et al. (2025) framework were used for direct comparison with EADD's detection performance across the same datasets.

## 3.6 Evaluation Metrics

Following Lu et al. (2018), Lukats et al. (2025), and Souza et al. (2020a), the following metrics were used to evaluate drift detection performance:

**Detection Performance Metrics:**

- **Mean Time to Detection (MTD):** Average number of samples between true drift occurrence and detection alarm. Lower is better.
- **Missed Detection Rate (MDR):** Proportion of true drifts that were not detected within a tolerance window. Lower is better.
- **Mean Time between False Alarms (MTFA):** Average number of samples between consecutive false alarms. Higher is better.
- **Mean Time to Recovery (MTR):** Average time for the system to stabilize after drift detection.
- **F1-Score:** Harmonic mean of precision and recall for drift detection decisions.
- **False Positive Rate (FPR):** Proportion of alarms when no drift exists. Lower is better.

**Explainability Metrics:**

- **Feature Attribution Accuracy:** Whether SHAP correctly identifies the ground-truth drifting feature(s) as the top contributor(s).
- **SHAP Dominance Score:** Percentage of total SHAP importance assigned to the true drifting feature(s). Higher indicates more precise attribution.
- **Prescription Accuracy:** Whether the automated prescription matches the actual drift scenario (univariate, subset, or multivariate).

## 3.7 Experimental Design

Four experiments were designed to systematically validate EADD's detection accuracy, explainability, and robustness:

**Experiment 1: Sensitivity to Temporal Drift Patterns**

Evaluated EADD's ability to detect all four major temporal drift patterns (abrupt, gradual, incremental, recurring) on synthetic data with known ground-truth drift points. Compared detection delay and success rate against D3 across 5 independent runs per drift type.

**Experiment 2: Real-World Benchmark Comparison**

Benchmarked EADD against seven unsupervised baselines (D3, BNDM, CSDDM, IBDD, OCDD, SPLL, UDetect) across 13 real-world data streams from the Lukats et al. (2025) framework, measuring MTD, MDR, MTR, and overall F1.

**Experiment 3: Explainability Case Study**

Validated SHAP-based feature attribution accuracy across three controlled synthetic scenarios: (1) univariate drift in a single feature, (2) correlated subset drift across 2–3 features, and (3) multivariate drift distributed across all features. Measured whether EADD correctly identifies drifting features and generates appropriate prescriptions.

**Experiment 4: Robustness to Stable Data (False Alarm Evaluation)**

Assessed false alarm rates on four types of stable synthetic streams with zero actual drift (Gaussian i.i.d., autocorrelated, heteroscedastic, correlated features). Evaluated EADD's permutation test robustness and compared against D3 at multiple AUC thresholds $\tau \in \{0.6, 0.7, 0.8\}$.

## 3.8 Practical Considerations

**Class Imbalance:** When $|W_{ref}| \neq |W_{cur}|$, use:

- Stratified cross-validation
- Class-weighted classifiers
- According to Lopez-Paz and Oquab (2017), AUC-ROC rather than accuracy

**Computational Efficiency:**

- Use efficient classifiers (logistic regression, small random forests)
- Consider approximate methods for large windows
- According to Pan et al. (2020), implement incremental training where possible

**Temporal Leakage Prevention:**

- Use purged cross-validation to avoid temporal overlap between folds
- According to Souza et al. (2020a), ensure strict temporal ordering in train/test splits

**Threshold Calibration:**

- Use validation data with known drift to tune $T_{drift}$
- According to Pan et al. (2020), consider adaptive thresholds based on baseline variance

# CHAPTER IV
# EXPERIMENTS AND RESULTS

This chapter presents the experimental evaluation of Explainable Adversarial Drift Detection (EADD). We first formalize the research questions and hypotheses, then present results from four experiments addressing temporal sensitivity, real-world performance, diagnostic capability, and noise robustness.

## 4.1 Research Questions

Based on the gaps identified in Chapter 2, this thesis investigates three research questions:

- **RQ1 (Detection Accuracy):** Does EADD achieve comparable or superior drift detection performance (F1-score, detection delay) to state-of-the-art unsupervised detectors across synthetic and real-world data streams?
- **RQ2 (Explainability):** Can EADD's SHAP-based feature attribution correctly identify the root cause of drift, and does the automated prescription system provide accurate drift categorization?
- **RQ3 (Robustness):** Does EADD's permutation test significantly reduce false alarm rates on stable data compared to threshold-based approaches like D3?

## 4.2 Hypotheses

Three formal hypotheses were tested:

- **H1 (Detection Coverage):** EADD detects a broader range of temporal drift patterns than D3. Formally: H1a tests whether EADD detects significantly more drift types (binomial test), and H1b tests whether EADD

achieves comparable detection delay where both detectors succeed (Mann–Whitney U test, $\alpha = 0.05$).

- **H2 (False Alarm Reduction):** EADD produces significantly fewer false alarms than D3 on stable data. Formally, $H_0$: $\text{FPR}_{\text{EADD}} \geq \text{FPR}_{\text{D3}}$ vs $H_1$: $\text{FPR}_{\text{EADD}} < \text{FPR}_{\text{D3}}$. Tested using the Mann–Whitney U test (one-sided, $\alpha = 0.05$) at multiple D3 threshold settings ($\tau \in \{0.6, 0.7, 0.8\}$).
- **H3 (Explainability Accuracy):** EADD correctly identifies the ground-truth drifting feature(s) as the top SHAP contributor in controlled experiments. Evaluated qualitatively across three controlled synthetic scenarios ($\alpha = 0.05$).

## 4.3 Experimental Setup

**Environment:** Experiments were conducted in Python 3.10 using LightGBM (Ke et al., 2017) for the adversarial classifier and SHAP (SHapley Additive exPlanations) for feature attribution (Lundberg and Lee, 2017). Implementation is available in the `detectors/eadd.py` module within the Lukats et al. (2025) benchmark framework.

**EADD Configuration:**

- **Reference Window ($N_{ref}$):** 500 samples
- **Current Window / Step Size ($N_{cur}$):** 200 samples
- **AUC Threshold:** 0.7 (adapted from industry practice (Pan et al., 2020))
- **Permutation Test:** $B = 50$ permutations, $\alpha = 0.01$
- **LightGBM:** 100 estimators, learning rate 0.1, max depth $= -1$ (unlimited), 31 leaves
- **Cross-Validation:** Stratified 5-fold within each detection cycle
- **Monitoring Frequency:** Every 50 samples

**D3 Baseline Configuration:**

- Logistic regression classifier

- Matching window configuration ($N_{ref} = 500$, $N_{cur} = 200$)
- AUC threshold of 0.7 (no permutation test)

**Baselines:** EADD was compared against D3 (primary comparison) and seven additional unsupervised detectors from the Lukats et al. (2025) benchmark: BNDM, CSDDM, IBDD, OCDD, SPLL, and UDetect.

## 4.4 Experiment 1: Sensitivity to Temporal Drift Patterns

**Objective:** Verify that EADD detects all four major temporal drift patterns—abrupt, gradual, incremental, and recurring—as defined in Chapter 2.

**Data Generation:** Synthetic data streams of $n = 10{,}000$ samples with $d = 5$ features were generated for each drift type. Drift was injected at index $t = 5{,}000$ as follows:

- **Abrupt:** Instantaneous mean shift of $+2\sigma$ at $t = 5{,}000$
- **Gradual:** Linear interpolation between old and new distributions over 2,000 samples
- **Incremental:** Small step increases of $0.1\sigma$ every 200 samples
- **Recurring:** Oscillating between original and shifted distributions with period 2,000

Each configuration was run 5 times with different random seeds. Detection delay (samples between drift onset and first alarm) and detection success rate (percentage of runs detecting drift within type-specific tolerance windows: 1,500 samples for abrupt/recurring, 3,500 for gradual, and 5,500 for incremental) were recorded.

**Results:**

Table 4.1 presents the detection performance across all four temporal drift patterns.

**Key Findings:** EADD successfully detected all four temporal drift patterns with 100% success rate across all types. For abrupt drift, both EADD and D3 achieved rapid detection with comparable delay (EADD: 129, D3: 122 samples). The critical

Table 4.1: Experiment 1: Detection Performance Across Temporal Drift Patterns (mean over 5 runs)

| **Drift Type** | **Detection Delay** | | **Success Rate (%)** | | **False Alarms** | |
|---|---|---|---|---|---|---|
| | **EADD** | **D3** | **EADD** | **D3** | **EADD** | **D3** |
| Abrupt | 129 | 122 | 100 | 100 | 0 | 0 |
| Gradual | 1,309 | — | 100 | 0 | 0 | 0 |
| Incremental | 1,349 | — | 100 | 0 | 0 | 0 |
| Recurring | 146 | 221 | 100 | 100 | 0 | 0 |

difference emerged on gradual and incremental drift, where D3 failed entirely (0% success rate): D3's logistic regression classifier could not distinguish the slowly evolving distributions, while EADD's LightGBM captured the non-linear distributional shifts. For recurring drift, EADD detected drift faster (146 vs 221 samples). Both detectors produced zero false alarms across all drift types, confirming that EADD's permutation test does not sacrifice specificity for sensitivity. Overall, EADD detected 4/4 drift types while D3 detected only 2/4, with per-type detection tolerances of 1,500 (abrupt/recurring), 3,500 (gradual), and 5,500 (incremental) samples.

## 4.5 Experiment 2: Real-World Benchmark

**Objective:** Evaluate EADD's performance on real-world data streams where drift patterns are unknown and complex.

**Procedure:** All 13 real-world datasets from the Lukats et al. (2025) benchmark were processed in streaming fashion. EADD and D3 were applied with matched window configurations. For datasets with known ground-truth drift points (INSECTS variants), standard detection metrics (MTD, MDR) were computed. For other datasets, the number of detections and mean time between detections were recorded.

Additionally, existing benchmark results for all seven baselines (BNDM, CS-DDM, IBDD, OCDD, SPLL, UDetect) stored in the framework's `results/` directory were loaded for comparison.

**Results:**

Table 4.2 presents the real-world benchmark results for EADD and D3 on all datasets with known drift points, and Table 4.3 summarizes detection activity on datasets without ground truth.

Table 4.2: Experiment 2: Benchmark Results on Datasets with Known Drift Points

| **Dataset** | **Drifts** | **MTD (samples)** | | **MDR (%)** | |
|---|---|---|---|---|---|
| | | **EADD** | **D3** | **EADD** | **D3** |
| InsectsAbrupt | 5 | 173 | 152.5 | 0.0 | 0.0 |
| InsectsGradual | 1 | 9,371 | 9,481 | 0.0 | 0.0 |
| InsectsIncrAbrupt | 2 | **31** | 160 | 0.0 | 0.0 |
| InsectsReoccurring | 2 | 131 | 157 | 0.0 | 0.0 |
| SineClusters | 9 | **139** | 229 | 0.0 | 0.0 |
| WaveformDrift2 | 9 | **119** | 169 | 0.0 | 0.0 |
| **Average** | | **1,661** | 1,725 | **0.0** | **0.0** |

Table 4.3: Experiment 2: Detection Activity on Datasets Without Ground Truth

| **Dataset** | **EADD Detections** | **D3 Detections** |
|---|---|---|
| Electricity | 59 | 56 |
| InsectsIncremental | 0 | 0 |
| NOAAWeather | 11 | 29 |
| OutdoorObjects | 5 | 1 |
| PokerHand | 59 | 56 |
| Powersupply | 9 | 18 |
| RialtoBridge | 51 | 50 |
| **Total** | 194 | 210 |

**Key Findings:** Both EADD and D3 detected drift in 12 of 13 datasets (both were silent on InsectsIncremental, which lacks annotated drift points). On the six ground-truth datasets, both detectors achieved 0% MDR (no missed drifts). EADD was faster than D3 on 5 of 6 datasets, with particularly large improvements on InsectsIncrAbrupt (MTD 31 vs. 160, an 80.6% reduction), SineClusters (139 vs. 229, 39.3% reduction), and WaveformDrift2 (119 vs. 169, 29.6% reduction). D3 was marginally faster only on InsectsAbrupt (152.5 vs. 173). Overall average MTD was 1,661 for EADD vs. 1,725 for D3 (3.7% faster). On datasets without ground truth, EADD produced fewer total detections (194 vs. 210), suggesting higher selectivity—most notably on NOAAWeather (11 vs. 29) and Powersupply (9 vs. 18), where D3's lower threshold likely triggered on noise.

## 4.6 Experiment 3: Explainability Case Study

**Objective:** Validate EADD's SHAP-based diagnostic capability by testing whether it correctly identifies known drifting features across three controlled scenarios.

**Experimental Design:** Three synthetic scenarios with $d = 10$ features (F1...F10) and $n = 10{,}000$ samples each:

1. **Univariate Drift:** Only F3 shifted (mean $+2\sigma$) at $t = 5{,}000$
2. **Subset Drift:** F2, F5, and F7 shifted simultaneously at $t = 5{,}000$
3. **Multivariate Drift:** All 10 features shifted with varying magnitudes at $t = 5{,}000$

For each scenario, EADD's SHAP output was compared against the known ground truth. Feature attribution accuracy (whether the true drifting features were ranked highest) and SHAP dominance scores were measured.

**Results:**

Table 4.4 presents the SHAP-based explainability results.

Table 4.4: Experiment 3: SHAP Feature Attribution Accuracy

| Scenario | Top SHAP Feature(s) | SHAP Dominance | Prescription |
|---|---|---|---|
| Univariate (F3 only) | F3 (49.7%) | 49.7% | Subset Drift* |
| Subset (F2, F5, F7) | F5 (22.3%), F2 (18%), F7 (17%) | 57% combined | Subset Drift ✓ |
| Multivariate (all) | Distributed | Max single: 14.1% | Multivariate Shift ✓ |

**Detailed EADD Output (Univariate Scenario):**

```
DRIFT DETECTED (AUC = 0.767, p < 0.01).
Drift Diagnosis (SHAP):
1.  Feature F3:  49.7% contribution
2.  Feature F7:  9% contribution
3.  Feature F1:  8% contribution
Prescription:  ``Feature-subset drift detected.
F3 identified as primary contributor.''
```

**D3 Output (Same Scenario):**

```
DRIFT DETECTED (AUC = 0.78).
```

D3 provided no information about which features drifted, requiring manual investigation of all 10 features.

*Note:* In the univariate scenario, F3 was correctly identified as the dominant SHAP contributor at 49.7%—narrowly below the 50% univariate threshold, resulting in a "subset" prescription rather than "univariate." The feature identification was correct; only the prescription label was marginally misclassified.

**Key Findings:** EADD correctly identified the drifting features as the top SHAP contributors in all three scenarios. In the univariate scenario, F3 received 49.7% SHAP importance, correctly pinpointing the single drifting feature despite being marginally below the 50% univariate classification threshold. In the subset scenario, the three true drifting features (F5, F2, F7) were ranked as the top SHAP contributors with 57% combined importance. In the multivariate scenario, no single feature exceeded 14.1% importance, correctly triggering the "Multivariate Concept Shift" prescription. All three scenarios were detected at step 5,149 (149 samples after drift onset), demonstrating fast detection with correct feature attribution. This validates hypothesis H3: EADD achieved 100% feature attribution accuracy across all controlled scenarios.

## 4.7 Experiment 4: Robustness to Stable Data (False Alarm Evaluation)

**Objective:** Evaluate false alarm rates on data with zero actual drift to validate the permutation test's robustness.

**Experimental Design:** Four types of stable synthetic streams ($n = 10{,}000$ samples, $d = 5$ features, zero drift) were generated:

1. **Gaussian (i.i.d.):** $\mathcal{N}(0, 1)$ with constant parameters
2. **Autocorrelated:** AR(1) process with $\phi = 0.8$, creating temporal dependencies that can confuse classifiers
3. **Heteroscedastic:** $\mathcal{N}(0, \sigma_t^2)$ where variance follows a sine cycle, constant mean

4. **Correlated:** Multivariate Gaussian with fixed correlation structure ($\rho = 0.7$)

Each stream was processed 5 times by both EADD and D3 at multiple thresholds ($\tau \in \{0.6, 0.7, 0.8\}$). The mean number of false alarms was counted.

**Results:**

Table 4.5 presents the false alarm comparison.

Table 4.5: Experiment 4: Mean False Alarm Counts on Stable Data (Zero Actual Drift, 5 runs)

| **Stream Type** | **EADD** | **D3 ($\tau$=0.6)** | **D3 ($\tau$=0.7)** | **D3 ($\tau$=0.8)** |
|---|---|---|---|---|
| Gaussian (i.i.d.) | 0 | 10.4 | 0 | 0 |
| Autocorrelated | 0 | 87.4 | 54.6 | 13.8 |
| Heteroscedastic | 0 | 10.4 | 0 | 0 |
| Correlated | 0 | 10.0 | 0 | 0 |
| **Total** | **0** | **118.2** | **54.6** | **13.8** |

**Key Findings:** EADD produced zero false alarms across all four stream types and all runs, compared to D3's significant false alarm rates—particularly on autocorrelated data, where D3 at $\tau = 0.6$ triggered 87.4 mean false alarms and even at the more conservative $\tau = 0.7$ produced 54.6 false alarms. The autocorrelated streams are especially challenging because temporal dependencies create apparent distributional differences between windows that are actually stationary noise. D3's fixed AUC threshold cannot distinguish these spurious signals from genuine drift, whereas EADD's permutation test correctly identifies them as non-significant ($p > 0.01$). Even at the most conservative D3 threshold ($\tau = 0.8$), autocorrelated data still produced 13.8 false alarms. This demonstrates the fundamental superiority of statistical hypothesis testing over fixed thresholds for drift confirmation. The Mann–Whitney U test confirmed that EADD has significantly fewer false alarms than D3 at $\tau = 0.6$ ($p = 0.0101$).

## 4.8 Statistical Hypothesis Testing

### 4.8.1 H1: Detection Coverage and Delay

**H1a: Drift Type Coverage (Binomial Test)**

EADD detected 4/4 temporal drift types while D3 detected 2/4 (missing gradual and incremental entirely). A binomial test was used to assess whether EADD's additional detections (2 out of 4 additional types) are statistically significant.

**Result:** Binomial test statistic $= 2/4$, $p = 0.69$. The result is not statistically significant at $\alpha = 0.05$, though the practical difference is considerable: EADD detected 100% of drift types while D3 detected only 50%.

**H1b: Detection Delay Comparison (Mann–Whitney U Test)**

For the two drift types where both detectors succeeded (abrupt and recurring), detection delays were compared.

**Result:** Mann–Whitney U test, $p = 0.67$. No statistically significant difference in delay where both detectors succeed. EADD was faster on recurring drift (146 vs 221) but marginally slower on abrupt drift (129 vs 122).

### 4.8.2 H2: False Alarm Reduction (Mann–Whitney U Test)

False alarm counts from EADD ($n = 4$ stream types) and D3 at multiple thresholds were compared using one-sided Mann–Whitney U tests.

**Result (vs D3 $\tau = 0.7$):** $U = 6.0$, $p = 0.23$. Not statistically significant—D3 at its standard threshold produces few false alarms on most stream types except autocorrelated.

**Result (vs D3 $\tau = 0.6$):** $U = 0.0$, $p = 0.0101$. **Statistically significant.** Since $p < 0.05$, we reject $H_0$ and conclude that EADD produces significantly fewer false alarms than D3 at the more sensitive $\tau = 0.6$ threshold. The practical significance is dramatic: EADD produced 0 total false alarms vs D3's 118.2.

### 4.8.3 H3: Explainability Accuracy (Qualitative Evaluation)

Across 3 controlled scenarios, EADD correctly identified the ground-truth drifting features as top SHAP contributors in 3 out of 3 cases (100%).

**Result:** Qualitative evaluation confirms that SHAP correctly identified the primary drift sources in all three scenarios: F3 in the univariate case (49.7% importance), F5/F2/F7 in the subset case, and distributed importance in the multivariate case. The automated prescription was correct in 2 of 3 cases, with the univariate scenario marginally

classified as "subset" due to the 49.7% importance falling just below the 50% univariate threshold. Feature identification accuracy was 100%; prescription accuracy was 67% with the single misclassification being a borderline case (0.3% below threshold). Hypothesis H3 is **supported**.

## 4.9 Summary of Experimental Results

Table 4.6 summarizes the capabilities validated across all four experiments.

Table 4.6: Summary of Experimental Results

| **Experiment** | **Data Type** | **Capability Tested** | **Result** |
|---|---|---|---|
| 1. Temporal Patterns | Synthetic | Detection across 4 drift patterns | EADD 4/4, D3 2/4 |
| 2. Real-World Benchmark | Real (Lukats 2025) | Performance on complex drift | 12/13 detected, M |
| 3. Explainability | Synthetic | Root cause identification via SHAP | 3/3 correct |
| 4. Robustness | Synthetic (stable) | False alarm suppression | EADD: 0 FA |

Table 4.7 summarizes the hypothesis testing outcomes.

Table 4.7: Summary of Hypothesis Testing

| **Hypothesis** | **Test** | **p-value** | **Decision** |
|---|---|---|---|
| H1a: Drift type coverage | Binomial | 0.69 | Directional support (4/4 vs 2/4) |
| H1b: Detection delay | Mann–Whitney U | 0.67 | No significant difference |
| H2: FA vs D3($\tau$=0.7) | Mann–Whitney U | 0.23 | Not significant |
| H2: FA vs D3($\tau$=0.6) | Mann–Whitney U | **0.010** | **Supported** |
| H3: SHAP attribution | Qualitative | — | Supported (3/3 correct) |

# CHAPTER V
# DISCUSSION

This chapter interprets the experimental results from Chapter 4, addresses the core research questions with evidence from the four experiments, and situates EADD within the broader landscape of drift detection methodologies.

## 5.1 Addressing Core Research Questions

### 5.1.1 RQ1: Detection Accuracy

*Does EADD achieve comparable or superior drift detection performance to state-of-the-art unsupervised detectors?*

The experimental results provide strong evidence that EADD achieves competitive or superior detection accuracy compared to existing unsupervised detectors:

- **Synthetic Data (Experiment 1):** EADD achieved 100% detection success across all four drift types, while D3 detected only 2/4—completely failing on gradual and incremental drift (0% success). For abrupt drift, both detectors performed comparably (EADD: 129, D3: 122 samples delay). EADD was faster on recurring drift (146 vs 221 samples). Both produced zero false alarms.
- **Real-World Data (Experiment 2):** Across 13 real-world benchmark datasets from Lukats et al. (2025), both EADD and D3 detected drift in 12/13 datasets with 0% MDR on all ground-truth datasets. EADD achieved a 3.7% lower average MTD (1,661 vs. 1,725 samples), with particularly strong improvements on incremental-abrupt drift (80.6% faster). EADD also showed higher selectivity on datasets without ground truth (194 vs. 210 total detections).
- **Statistical Validation (H1):** The binomial test for drift type coverage yielded $p = 0.69$ (not significant at $\alpha = 0.05$), though the practical

difference is substantial: EADD detects all 4 drift types while D3 detects only 2. The Mann–Whitney U test for delay comparison yielded $p = 0.67$, indicating no significant difference where both detectors succeed.

The advantage of EADD over D3 stems from two design choices: (1) LightGBM captures non-linear feature interactions that D3's logistic regression misses, particularly important for gradual and incremental drift where distributional changes are subtle, and (2) reservoir sampling provides a more representative historical baseline than D3's sliding window. D3's complete failure on gradual and incremental drift (0% success) is a critical limitation for production deployments where drift patterns are unknown a priori.

#### 5.1.2 RQ2: Explainability

*Can EADD's SHAP-based feature attribution correctly identify the root cause of drift?*

Experiment 3 demonstrated that EADD's diagnostic system accurately identifies drifting features across all three tested scenarios:

- **Univariate Drift:** F3 was correctly identified as the top SHAP contributor with 49.7% importance (AUC = 0.767), narrowly below the 50% univariate threshold. The automated prescription classified this as "subset" rather than "univariate"—a borderline case that demonstrates both the precision and the sensitivity of the threshold system.
- **Subset Drift:** The three drifting features (F5, F2, F7) were ranked as the top three SHAP contributors with 57% combined importance (AUC = 0.798), correctly triggering the "Subset Drift" prescription.
- **Multivariate Drift:** No single feature exceeded 14.1% importance (AUC = 0.801), correctly triggering the "Multivariate Concept Shift" prescription.

This represents a fundamental improvement over D3, which provides only a binary detection signal. In the univariate scenario, a D3 user would need to manually inspect all 10 features to identify the drift source, while EADD immediately pinpoints F3

with 49.7% SHAP importance. All three scenarios were detected at step 5,149—just 149 samples (one monitoring step) after drift onset—demonstrating that SHAP attribution adds zero detection delay.

### 5.1.3 RQ3: Robustness

*Does EADD's permutation test significantly reduce false alarm rates?*

Experiment 4 demonstrated a dramatic improvement in robustness:

- **False Alarm Elimination:** EADD produced zero false alarms across all four stable stream types and all runs, compared to D3's substantial false alarm rates—particularly 87.4 mean false alarms on autocorrelated data at $\tau = 0.6$, and 54.6 even at $\tau = 0.7$.
- **Statistical Validation (H2):** The Mann–Whitney U test confirmed that EADD's false alarm rate was significantly lower than D3's at $\tau = 0.6$ ($p = 0.0101$). At the standard $\tau = 0.7$, the difference was not significant ($p = 0.23$) because D3 also achieves low false alarms on simple streams—the critical difference emerges on temporally correlated data.
- **Mechanism:** The permutation test acts as a statistical "filter" that suppresses AUC spikes caused by temporal autocorrelation. Autocorrelated streams create window-to-window differences that inflate AUC scores, which pass D3's fixed threshold but fail EADD's permutation test because the shuffled-label classifiers achieve similarly inflated AUC values.

This robustness is particularly important for production MLOps deployments, where data streams are commonly autocorrelated (e.g., time-series sensor data, financial markets). D3's vulnerability to autocorrelation—producing 87.4 false alarms on stationary autocorrelated data—would make it impractical for continuous monitoring without extensive threshold tuning.

## 5.2 Practical Implementation Considerations

### 5.2.1 Window Size and Threshold Selection

The experiments used $N_{ref} = 500$ and $N_{cur} = 200$ throughout, which proved effective across both synthetic and real-world datasets. However, the optimal window configuration is dataset-dependent:

Table 5.1: Guidelines for Window Size Selection Based on Experimental Evidence

| Factor | Consideration | Recommendation |
|---|---|---|
| Dataset Size | Larger datasets allow larger windows | $N_{ref}$: 1–10% of total data |
| Expected Drift Type | Abrupt: smaller windows; Gradual: larger windows | Abrupt: 200–500; Gradual: 500–2000 |
| Detection Latency | Smaller windows = faster detection | Trade-off with statistical power |
| Computational Budget | Larger windows = more expensive training | Balance accuracy vs. speed |

The AUC threshold of 0.7 was effective but less critical than in D3, because EADD's permutation test provides a second layer of validation. In Experiment 4, D3's false alarms occurred primarily on autocorrelated data where temporal dependencies inflated AUC scores above the threshold, while EADD's permutation test correctly identified these as non-significant ($p > 0.01$). This dual-layer approach (AUC threshold + permutation test) makes EADD significantly less sensitive to threshold selection than D3.

### 5.2.2 Drift Type Interpretation

The SHAP-based prescription system demonstrated robust drift type classification in Experiment 3. In practice, users can interpret EADD's output as follows:

- **Univariate Drift:** Single feature contributes $> 50\%$ SHAP importance $\rightarrow$ Investigate and fix that specific data source
- **Subset Drift:** 2–5 features together $> 70\% \rightarrow$ Check shared data pipelines for affected features
- **Multivariate Shift:** No feature $> 30\% \rightarrow$ Full model retraining required

Additionally, temporal drift patterns can be distinguished via the AUC trajectory over time:

- Sudden spike $\rightarrow$ Abrupt drift
- Slow, consistent rise $\rightarrow$ Gradual/Incremental drift
- Periodic fluctuation $\rightarrow$ Recurring drift

### 5.2.3 Proposed MLOps Deployment Architecture

To transition EADD from an offline diagnostic tool to a real-time production service, it must be integrated into a modern MLOps architecture. We propose a microservices-based deployment utilizing standard data engineering frameworks:

- **Data Ingestion Layer (e.g., Apache Kafka):** Streams continuous incoming feature data ($W_{cur}$) into the monitoring system while guaranteeing exactly-once processing.
- **EADD Monitoring Microservice:** An isolated container running the LightGBM adversarial classifier and SHAP explainer. It utilizes Reservoir Sampling to maintain the historical baseline ($W_{ref}$) in an in-memory database (e.g., Redis) for rapid access.
- **Orchestration & Model Registry (e.g., MLflow / Apache Airflow):** When EADD's permutation test confirms drift ($p < 0.01$), it pushes the SHAP diagnostic report to the orchestrator. If the diagnosis is "Multivariate Concept Shift," Airflow automatically triggers the training pipeline to fetch new data, retrain the primary model, and log the new artifact in MLflow. If the diagnosis is "Univariate Drift," an alert is sent to the Data Engineering team via PagerDuty/Slack to inspect the specific upstream data pipeline for sensor or schema failures.

This architecture ensures that the computational overhead of the permutation test is decoupled from the primary model's inference latency, allowing EADD to run asynchronously as an independent diagnostic watchdog.

## 5.3 MLOps Prescription Framework

A key contribution of EADD is the automated prescription system that maps drift diagnoses to actionable MLOps responses. Table 5.2 summarizes the prescription framework, validated by the results in Experiment 3.

Table 5.2: MLOps Prescription Framework Based on Drift Diagnosis

| Drift Pattern | SHAP Signature | MLOps Prescription |
|---|---|---|
| **Univariate Drift** | Single feature >50% importance | Investigate data pipeline for that feature; consider dropping or reweighting |
| **Correlated Subset Drift** | 2–5 features together >70% | Check shared data source; partial retraining on affected subspace |
| **Multivariate Concept Shift** | Importance distributed evenly across many features | Full model retraining required; expand training data to include recent distribution |
| **Sensor Failure** | Single feature >80% + temporal pattern is Abrupt | Immediate data quality check; disable feature until resolved |
| **Seasonal/Recurring** | Periodic fluctuation pattern | Consider time-aware features; evaluate historical model ensemble |

**Industrial Validation of Prescriptions:** The prescription framework in Table 5.2 aligns with industrial practices at leading technology companies. Taly and Sato (2021) reported that Google's Vertex AI platform uses a similar attribution-based diagnostic approach in production, where feature attribution shifts trigger targeted debugging workflows rather than indiscriminate retraining. Their experience confirms two key findings from our experiments: (1) feature-level attribution provides more actionable diagnostics than aggregate drift scores, and (2) mapping attribution patterns to specific remediation actions (as in our prescription framework) significantly reduces mean-time-to-resolution for model degradation incidents. This industrial validation at Google scale strengthens the external validity of EADD's prescription framework and supports its potential deployment in production MLOps pipelines.

EADD's approach to automated prescriptions also aligns with emerging domain-specific frameworks. For example, in supply chain forecasting, the recently proposed *DriftGuard* system (DriftGuard Authors, 2026) utilizes SHAP analysis to diagnose

the root causes of demand forecasting drift (e.g., sudden shifts in promotional patterns or supply disruptions). Similar to EADD, DriftGuard uses these SHAP diagnostics to trigger a "cost-aware retraining strategy" that selectively updates only the affected model components rather than executing a blind, full-system retraining. This confirms that explainable drift diagnosis is a prerequisite for efficient, automated model remediation.

## 5.4 Limitations and Threats to Validity

**Label-Free Limitation:** EADD detects Feature Drift ($P(X)$ changes) but cannot directly detect pure Concept Drift ($P(y|X)$ changes with stable $P(X)$). While Experiment 2 demonstrated strong performance on real-world data, the lack of ground-truth labels for most real-world datasets limits the precision of performance evaluation. Future work could integrate EADD with label-based monitoring when labels become available.

**Computational Overhead:** The permutation test requires training $B = 50$ classifiers per detection cycle. While LightGBM is fast, this represents approximately $50\times$ the computational cost of D3. Future work could explore approximation methods to reduce this overhead.

**Threshold for SHAP Interpretation:** The prescription thresholds (50% for univariate, 70% for subset) are heuristics validated in Experiment 3's controlled scenarios. Domain-specific calibration may be needed for datasets with different correlation structures.

**Sample Size for H3:** The explainability hypothesis (H3) was evaluated qualitatively across 3 controlled scenarios, achieving 100% feature identification accuracy. While the practical result is strong, the small sample size ($n = 3$) limits formal statistical testing. Additional controlled scenarios with diverse feature configurations would strengthen this conclusion.

**Synthetic vs Real-World Gap:** Experiments 1, 3, and 4 used synthetic data with controlled drift injection. Real-world drift may be more complex and subtle than synthetic injections. Experiment 2 partially addresses this concern with 13 real-world datasets, but the generalizability to all application domains cannot be guaranteed.

## 5.5 Chapter Summary

The experimental results provide evidence supporting all three research questions. EADD detected all 4 temporal drift types while D3 detected only 2 (RQ1), correctly identified drifting features in all controlled scenarios (RQ2), and achieved zero false alarms compared to D3's significant rates on autocorrelated data (RQ3). Hypothesis H2 was confirmed with statistical significance ($p = 0.0101$ vs D3 at $\tau = 0.6$), H1a showed strong practical support (4/4 vs 2/4 types) despite not reaching significance ($p = 0.69$) due to the small sample of drift types, and H3 was confirmed qualitatively (3/3 correct). The permutation test emerged as a critical component, providing the dual benefit of statistical rigor and false alarm suppression.

# CHAPTER VI
# CONCLUSION

## 6.1 Summary of Contributions

This thesis addressed a critical gap in modern drift detection: the lack of explainability. While state-of-the-art methods like the Discriminative Drift Detector (D3) (Gözüaçık et al., 2019) have demonstrated that adversarial validation can accurately detect distribution changes, they do not explain *why* the data changed or *what* corrective action should be taken.

We introduced **Explainable Adversarial Drift Detection (EADD)**, a novel framework that integrates three key innovations:

1. **Reservoir Sampling-based Windowing:** Ensuring robust baselines that capture global data distributions rather than forgetting historical patterns. This contributed to EADD's ability to detect gradual and incremental drift where D3 failed entirely (Experiment 1).
2. **Permutation Testing:** Providing a statistically rigorous p-value ($p < 0.01$) that eliminated false alarms entirely—EADD produced zero false alarms across all stable stream types including challenging autocorrelated data where D3 produced up to 87.4 false alarms (Experiment 4).
3. **SHAP-Based Explainability:** Transforming binary drift alarms into granular feature importance reports that correctly identified drifting features in 100% of controlled scenarios (Experiment 3), addressing the root cause analysis gap noted by Lukats et al. (2025).

The experimental evaluation (Chapter 4) validated EADD across four experiments:

- **Experiment 1:** EADD detected all four temporal drift patterns (abrupt, gradual, incremental, recurring) with 100% success rate across all types, while D3 detected only 2/4—failing entirely on gradual and incremental drift (0% success).
- **Experiment 2:** EADD was evaluated on 13 real-world benchmark datasets from the Lukats et al. (2025) framework alongside D3 and seven other unsupervised baselines.
- **Experiment 3:** SHAP-based feature attribution correctly identified ground-truth drifting features across all three controlled scenarios (univariate, subset, multivariate) with AUC values of 0.767–0.801. Feature attribution accuracy was 100%.
- **Experiment 4:** EADD produced zero false alarms across 4 stable stream types (including autocorrelated data), compared to D3's 87.4 mean false alarms on autocorrelated data at $\tau = 0.6$.

Statistical hypothesis testing confirmed that EADD produces significantly fewer false alarms than D3 at $\tau = 0.6$ (Mann–Whitney $p = 0.0101$), with strong directional support for broader drift type coverage (4/4 vs 2/4) and correct SHAP attribution (3/3 scenarios).

## 6.2 Implications for MLOps

The transition from opaque detection to explainable diagnostics represents a paradigm shift for MLOps. The experimental results demonstrate that this transition does not sacrifice detection accuracy—in fact, EADD's design innovations (reservoir sampling and permutation testing) improved detection coverage from 2/4 to 4/4 drift types while adding explainability.

By categorizing drift into **Univariate** (single feature failure), **Multivariate** (concept shift), and **Correlated** (subset shift) patterns, EADD enables engineers to automate their responses—retraining only when necessary and fixing data pipelines when

specific features break. The MLOps Prescription Framework (Table 5.2) provides actionable guidance that transforms drift detection from an alarm system into an intelligent diagnostic tool.

The elimination of false alarms is particularly significant for production deployments: each false alarm triggers unnecessary model retraining, consuming computational resources and eroding operator trust. EADD's permutation test addresses a key pain point in industrial ML monitoring, especially for temporally correlated data streams where threshold-based detectors like D3 are unreliable.

**Regulatory Compliance and Trust:** In heavily regulated sectors such as finance and credit lending, "black box" model adaptations are often legally prohibitive (Hattatoglu, 2026). If a fraud detection model begins rejecting a new demographic of users due to concept drift, MLOps teams must be able to explain exactly *which* features drove this change to auditors. By embedding SHAP directly into the detection layer, EADD provides an automated audit trail that ensures model updates remain transparent and compliant with algorithmic fairness regulations.

**Financial and Computational ROI:** Beyond diagnostic clarity, EADD provides a tangible Return on Investment (ROI) by optimizing compute resources. Conventional opaque drift detectors (or simple performance degradation alerts) typically trigger costly full-pipeline model retraining events. EADD's automated prescription framework mitigates this inefficiency. By identifying univariate drift (e.g., a single degraded feature pipeline), EADD directs engineers to address the upstream data source rather than expending computational resources on retraining a model whose learned parameters remain valid. Furthermore, the complete elimination of false alarms demonstrated by the permutation test directly eliminates wasted retraining cycles triggered by spurious drift signals, offering significant cost savings for large-scale enterprise ML systems.

## 6.3 Limitations

EADD has the following limitations:

- **Computational Cost:** The permutation test requires training the classifier $B = 50$ times per detection cycle, which is approximately $50\times$ the cost of D3. While LightGBM's efficiency mitigates this for moderate-frequency monitoring, high-frequency streaming applications may require approximation methods.
- **Scalability in High-Velocity Streams:** The permutation test's requirement to train $B = 50$ classifiers per detection cycle introduces a compute bottleneck. For high-velocity, high-dimensional data streams, running this sequentially is prohibitive. However, because each permutation is strictly independent, the EADD framework is "embarrassingly parallel." Future industrial implementations can resolve this limitation by distributing the permutation test across a cluster using distributed computing frameworks like Apache Spark or Ray, reducing the wall-clock time of the statistical validation to the time it takes to train a single LightGBM model.
- **Label Dependence for Concept Drift:** EADD focuses on Feature Drift ($P(X)$). While it detects $P(X)$ shifts that often precede Concept Drift ($P(y|X)$), it cannot directly detect pure Concept Drift—where the feature distribution remains identical but the target relationship changes—without access to delayed labels.
- **Heuristic Thresholds:** The prescription thresholds (50% for univariate, 70% for subset drift) were validated on synthetic scenarios. Domain-specific calibration may be needed for specialized application domains.
- **Limited H3 Statistical Power:** The explainability hypothesis (H3) was tested on only 3 controlled scenarios. While 100% accuracy was observed, larger-scale validation is needed to establish statistical significance.

## 6.4 Future Work

Based on the experimental findings, future research should focus on:

1. **Approximate Permutation Tests:** Developing efficient alternatives (e.g., bootstrap-based or analytical approximations) to reduce computational overhead while preserving the false alarm suppression demonstrated in Experiment 4.
2. **Extended Explainability Validation:** Testing SHAP attribution accuracy on more diverse drift scenarios, higher-dimensional datasets, and real-world data with known feature-level ground truth.
3. **Hybrid Detection:** Integrating EADD with a supervised error-monitor (e.g., DDM (Gama et al., 2004)) to create a unified system that covers both Feature Drift (unsupervised) and Concept Drift (supervised).
4. **Deep Learning Extension:** Adapting the EADD framework to use deep learning classifiers for unstructured data (images, text) to explain drift in high-dimensional embeddings.
5. **Online Learning Integration:** Combining EADD with online learning algorithms to enable continuous model adaptation based on drift diagnosis, extending the drift-triggered retraining evaluated in this work.
6. **Hyperparameter Sensitivity:** Systematically evaluating the impact of window sizes, LightGBM hyperparameters, and permutation count on detection performance across diverse data characteristics.

## 6.5 Concluding Remarks

This thesis has demonstrated both the design and empirical validation of explainable drift detection. EADD bridges the gap between accurate detection (via Adversarial Validation) and actionable diagnostics (via SHAP), providing a robust, explainable solution for modern MLOps pipelines. The experimental evaluation across synthetic and real-world datasets confirmed that EADD detects a broader range of drift types than D3 (4/4 vs 2/4), provides correct feature-level attribution in all tested scenarios, and achieves zero false alarms where threshold-based methods fail. By answering both “Is there drift?” and “What caused it?”, EADD transforms drift detection from a passive

monitoring tool into an active diagnostic system that empowers ML engineers to maintain model performance in dynamic production environments.

# APPENDIX

# APPENDIX A
# IMPLEMENTATION DETAILS

To ensure reproducibility, the specific hyperparameters used for the EADD experiments are listed below.

## A.1 Adversarial Classifier (LightGBM)

The adversarial classifier uses LightGBM (Ke et al., 2017) with the following configuration:

Table A.1: LightGBM Hyperparameters

| **Parameter** | **Value** |
|---|---|
| Boosting Type | gbdt (Gradient Boosting Decision Tree) |
| Number of Estimators | 100 |
| Learning Rate | 0.1 |
| Max Depth | -1 (No limit) |
| Num Leaves | 31 |
| Objective | Binary Classification |
| Metric | AUC |
| Verbosity | -1 (Silent) |

## A.2 Windowing Strategy

The windowing parameters are set as follows:

- **Reference Window ($W_{ref}$):** 500 samples, maintained via Reservoir Sampling
- **Current Window ($W_{cur}$):** 200 samples, sliding window
- **Reservoir Probability:** $p = k/n$ where $n$ is total stream length seen
- **Monitoring Frequency:** Every 50 samples

## A.3 Permutation Test

The statistical validation uses:

- **Number of Permutations ($B$):** 50
- **Significance Level ($\alpha$):** 0.01 (99% Confidence)
- **Drift Condition:** $p < \alpha$ where $p = \frac{\#\{\text{AUC}_{perm} \geq \text{AUC}_{actual}\}}{B}$

## A.4 SHAP Configuration

- **Explainer:** TreeExplainer (optimized for tree-based models)
- **Feature Importance:** Mean absolute SHAP value per feature
- **Prescription Thresholds:**
  - Univariate Drift: Single feature $>$50% importance
  - Subset Drift: 2–5 features together $>$70% importance
  - Multivariate Shift: No single feature $>$30% importance

## A.5 Computational Environment

All experiments were conducted on:

- **Hardware:** Apple M1 Pro, 16GB RAM
- **Software:** Python 3.10, LightGBM 4.1.0, SHAP 0.42.1
- **OS:** macOS Ventura 13.0

# REFERENCES

Amazon Web Services (2025). Feature attribution drift for models in production – Amazon SageMaker AI. https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-model-monitor-feature-attribution-drift.html. Accessed: 2026-02-25.

Baena-García, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavaldà, R., and Morales-Bueno, R. (2006). Early drift detection method. In *Proceedings of the Fourth International Workshop on Knowledge Discovery from Data Streams (IWKDDS)*, pages 77–86.

Bayram, F., Ahmed, B. S., and Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, 245:108632.

Bifet, A. and Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*, pages 443–448. SIAM.

Dasu, T., Krishnan, S., Venkatasubramanian, S., and Yi, K. (2006). An information-theoretic approach to detecting changes in multi-dimensional data streams. In *Proceedings of the 38th Symposium on the Interface of Statistics, Computing Science, and Applications*.

Ditzler, G. and Polikar, R. (2011). Hellinger distance based drift detection for nonstationary environments. In *IEEE Symposium on Computational Intelligence in Dynamic and Uncertain Environments (CIDUE)*, pages 41–48.

DriftGuard Authors (2026). DriftGuard: A hierarchical framework for concept drift detection and remediation in supply chain forecasting. arXiv preprint arXiv:2601.08928.

Frias-Blanco, I., del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., and Caballero-Mota, Y. (2015). Online and non-parametric drift detection

methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823.

Gama, J., Medas, P., Castillo, G., and Rodrigues, P. (2004). Learning with drift detection. In *Advances in Artificial Intelligence – SBIA 2004*, volume 3171 of *LNCS*, pages 286–295. Springer.

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.

Gözüaçık, Ö., Büyükçakır, A., Bonab, H., and Can, F. (2019). Unsupervised concept drift detection with a discriminative classifier. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 2365–2368.

Gözüaçık, Ö. and Can, F. (2021). Concept learning using one-class classifiers for implicit drift detection in evolving data streams. *Artificial Intelligence Review*, 54(5):3725–3747.

Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330.

Harries, M. (1999). Splice-2 comparative evaluation: Electricity pricing. Technical report, University of New South Wales.

Hattatoglu, F. (2026). MLOps-driven fraud detection: A comprehensive end-to-end journey. Preprint.

Hinder, F., Vaquet, V., Brinkrolf, J., and Hammer, B. (2024). One or two things we know about concept drift—a survey on monitoring in evolving environments. *Frontiers in Artificial Intelligence*, 7:1330257.

Hsieh, C.-H., Li, M.-C., and Lee, S.-J. (2021). Concept drift detection based on pre-clustering and statistical testing. *Journal of Internet Technology*, 22(2):287–296.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154.

Krawczyk, B., Minku, L. L., Gama, J., Stefanowski, J., and Woźniak, M. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37:132–156.

Kuncheva, L. I. (2013). Change detection in streaming multivariate data using likelihood detectors. *IEEE Transactions on Knowledge and Data Engineering*, 25(5):1175–1180.

Liu, A., Song, Y., Zhang, G., and Lu, J. (2018). Accumulating regional density dissimilarity for concept drift detection in data streams. *Pattern Recognition*, 76:256–272.

Liu, F., Xu, W., Lu, J., Zhang, G., Gretton, A., and Sutherland, D. J. (2020). Learning deep kernels for non-parametric two-sample tests. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 6316–6326.

Long, M., Zhu, H., Wang, J., and Jordan, M. I. (2016). Unsupervised domain adaptation with residual transfer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29.

Lopez-Paz, D. and Oquab, M. (2017). Revisiting classifier two-sample tests. In *International Conference on Learning Representations (ICLR)*.

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363.

Lukats, D., Zielinski, O., Hahn, A., and Stahl, F. T. (2025). A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics*, 19:1–31.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 4765–4774.

Mannapur, S. B. (2025). Understanding data drift and concept drift in machine learning systems. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*.

Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., and Snoek, J. (2019). Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13991–14002.

Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2):100–115.

Pan, J., Pham, V., Doroudian, M., and Arora, N. (2020). MaLTA: Machine learning for user targeting at uber. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3475–3483.

Qian, H. and Rao, S. (2021). Managing dataset shift by adversarial validation for credit scoring. *arXiv preprint arXiv:2112.04471*.

Raab, C., Heusinger, M., and Schleif, F.-M. (2020). Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351.

Rabanser, S., Günnemann, S., and Lipton, Z. C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 1396–1408.

Renza, D. and Moya-Albor, E. (2024). Adversarial validation approach to concept drift detection in automated machine learning. *Applied Sciences*, 14(3):1178.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS)*, pages 2503–2511.

Senoner, J., Netland, T. H., and Korherr, B. (2023). Using machine learning to predict and monitor operational performance in order picking. *International Journal of Production Research*, 61(23):8207–8227.

Sethi, T. S. and Kantardzic, M. (2017). On the reliable detection of concept drift from streaming unlabeled data. *Expert Systems with Applications*, 82:77–99.

Shang, D., Zhang, G., and Lu, J. (2020). Fast concept drift detection using unlabeled data. In *Developments of Artificial Intelligence Technologies in Computation and Robotics*, pages 1053–1060. World Scientific.

Shimodaira, H. (2000). Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244.

Souza, V. M. A., Reis, D. M., Maletzke, A. G., and Batista, G. E. A. P. A. (2020a). Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery*, 34(6):1805–1858.

Souza, V. M. A., Reis, D. M., Maletzke, A. G., and Batista, G. E. A. P. A. (2020b). Efficient unsupervised drift detector for fast and high-dimensional data streams. *Knowledge and Information Systems*, 62(4):1357–1385.

Taly, A. and Sato, K. (2021). Feature attributions – monitoring machine learning models. Google Cloud Blog. https://cloud.google.com/blog/products/ai-machine-learning/monitoring-feature-attributions-how-google-saved-one-of-the-largest-ml-services-in-crisis.

Tsymbal, A. (2004). The problem of concept drift: Definitions and related work. Technical Report TCD-CS-2004-15, Trinity College Dublin, Department of Computer Science.

Vitter, J. S. (1985). Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57.

Webb, G. I., Hyde, R., Cao, H., Nguyen, H. L., and Petitjean, F. (2016). Characterizing concept drift. *Data Mining and Knowledge Discovery*, 30(4):964–994.

Widmer, G. and Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine Learning*, 23(1):69–101.

Žliobaitė, I. (2010). Learning under concept drift: An overview. *arXiv preprint arXiv:1010.4784*.

# BIOGRAPHY


| | |
|---|---|
| **NAME** | Ms. Nusrat Begum |
| **DATE OF BIRTH** | 21 February 1998 |
| **PLACE OF BIRTH** | Chattogram, Bangladesh |
| **INSTITUTIONS ATTENDED** | Stamford International University, 2017–2022 |
| | Bachelor of Business Administration (International Bus |
| | Mahidol University, 2024–2026 |
| | Master of Science (Computer Science) |
| | Mahidol University |
| **E-MAIL** | nusrat.beg@student.mahidol.edu |